

گزارش فنی و تحلیلی پروژه

کاپوت

موتور قیمت‌گذاری و جست‌وجوی بازار خودروی دست‌دوم ایران

تحلیل کامل مسئله، معماری، خط لوله داده، سامانه کراول چهار منبعی، نرمال‌سازی، مدل یادگیری ماشین قیمت، نقش هوش مصنوعی، تصمیمات مهندسی و ارزش افزوده نسبت به سامانه‌های موجود.

۷,۶%	۲,۵۹۸	۴	۲۲,۸۲۰
خطای میانه مدل قیمت	قیمت بازیابی‌شده از «توافقی»	منبع داده مستقل	آگهی خودروی دست‌دوم

محدوده گزارش: کل مخزن — backend (Python ۳/۱۲ / FastAPI / scikit-learn) و frontend (React ۱۹ / TypeScript / Tailwind)

مبنای اعداد: تمام آمار این گزارش مستقیماً از پیکره داده و کد اجراشده استخراج و بازاندازه‌گیری شده است

تاریخ گزارش: ۱۳ شهریور ۱۴۰۵ — ۰۴-۰۹-۲۰۲۶

۱ خلاصه مدیریتی	۱۲ جست‌وجو: اول بازیابی، بعد رتبه‌بندی
۲ مسئله: سه پرسشی که هیچ سایتی پاسخ نمی‌دهد	۱۳ فیلترها: در دوم ورود به جست‌وجو
۳ راهکار کاپوت در یک نگاه	۱۴ ارزش‌گذاری خودروی کاربر
۴ معماری فنی و نقشه مازول‌ها	۱۵ API، کارایی و لایه رابط کاربری
۵ لایه کراول: چهار منبع، یک قرارداد	۱۶ تصمیمات معماری و دلیل هرکدام
۶ کنترل نرخ تطبیقی و اخلاق کراول	۱۷ فهرست کامل ویژگی‌های محصول
۷ نرمال‌سازی: از رشته نمایشی تا رکورد تایپ‌شده	۱۸ ارزش افزوده نسبت به پروژه‌های مشابه
۸ هویت بین‌منبعی و تشخیص آگهی تکراری	۱۹ مرجع کامل دستورات ترمینال
۹ موتور قیمت منصفانه	۲۰ کیفیت مهندسی، تست و استقرار
۱۰ موتور سلامت خودرو	۲۱ محدودیت‌ها و مسیر پیش رو
۱۱ هوش مصنوعی در کاپوت: سه کاربرد، یک قرارداد	۲۲ پیوست: جدول کامل آمار پیکره

درباره این گزارش

این سند از روی **کد اجراشده** نوشته شده، نه از روی مستندات پروژه. هر عددی که در ادامه می‌آید یا مستقیماً با پرس‌وجو از پایگاه‌داده پیکره (`backend/data/cars.db`) به دست آمده، یا با اجرای دوباره همان اسکریپت آموزش و ارزیابی که در مخزن هست. جایی که عدد اندازه‌گیری‌شده با عددی که در `README` پروژه ثبت شده تفاوت دارد، هر دو گزارش شده‌اند و دلیل اختلاف توضیح داده شده است.

روش اندازه‌گیری

آمار پیکره با پرس‌وجوی مستقیم SQL روی `cars.db`؛ خطای مدل قیمت با اجرای دوباره `pricing.train()` روی همان تقسیم تصادفی ۸۰/۲۰ با بذر ثابت؛ و کیفیت جست‌وجو با اجرای `python -m tests.eval_search` در دو حالت خط‌مبنا و فعلی. هیچ‌فایلی از پروژه در جریان این اندازه‌گیری‌ها بازنویسی نشد.

۱ خلاصه مدیریتی

کاپوت یک موتور تجمیع، قیمت‌گذاری و جست‌وجوی معنایی برای بازار خودروی دست‌دوم ایران است. برخلاف سایت‌های آگهی که «آنچه فروشنده نوشته» را بازنشر می‌کنند، کاپوت روی هر آگهی سه قضاوت مستقل می‌سازد: **قیمت واقعی بازار، امتیاز سلامت و تناسب با نیاز خریدار** – و مهم‌تر از همه، برای آگهی‌هایی که اصلاً قیمت ندارند («توافقی») هم عدد تولید می‌کند.

۹۳٫۲٪ دقت@۵ جست‌وجوی فارسی	۰٫۹۲۹ ضریب تعیین R^2 روی لگاریتم قیمت	۷٫۶٪ خطای میانه (median APE)	۲٫۵۹۸ آگهی بدون قیمت که قیمت گرفتند
--------------------------------------	---	--	---

سه دستاورد کلیدی

۳ هوش مصنوعی بدون وابستگی هر مسیر هوش مصنوعی یک مسیر جایگزین قطعی دارد. پاسخ‌های مدل روی دیسک کش می‌شوند و مسیر درخواست هرگز منتظر مدل نمی‌ماند. برنامه با شبکه قطع، کاملاً درست کار می‌کند – و پیکره فعلی دقیقاً در همین حالت ساخته شده است.	۲ بازیابی قبل از رتبه‌بندی نسخه اول هیچ مرحله بازیابی نداشت و کل پیکره را رتبه‌بندی می‌کرد؛ نتیجه این بود که «سراتو» یک BMW برمی‌گرداند. افزودن مرحله بازیابی، دقت@۵ را روی همین پیکره از ۴۲٫۱٪ به ۹۳٫۲٪ رساند.	۱ قیمت‌گذاری آگهی بی‌قیمت ۲٫۶۲۱ آگهی از ۲۲٫۸۲۰ آگهی پیکره هیچ قیمتی منتشر نمی‌کنند. در همه سایت‌های موجود این‌ها غیرقابل جست‌وجو، غیرقابل فیلتر و غیرقابل مقایسه‌اند. کاپوت برای ۲٫۵۹۸ تایی آن‌ها برآورد قیمت با بازه اطمینان تولید می‌کند و آن‌ها را در همان صف رتبه‌بندی و فیلتر قیمت قرار می‌دهد.
---	--	--

عدد پشت هر ادعا

ادعا	سنجه	مقدار اندازه‌گیری شده	مرجع در کد
قیمت‌گذاری دقیق است	خطای میانه نسبی روی داده کنارگذاشته	۷٫۶٪	pricing.train
قیمت‌گذاری دقیق است	MAPE روی داده کنارگذاشته	۱۴٫۴٪	pricing.train
مدل قیمت واریانس بازار را می‌گیرد	R^2 روی لگاریتم قیمت	۰٫۹۲۹	pricing.train
جست‌وجو خودروی درست را می‌آورد	precision@۵ روی ۳۸ پرسش برجسب‌خورده	۹۳٫۲٪	tests/eval_search.py
پیکره واقعاً چندمنبعی است	تعداد منابع فعال / آگهی	۲۲٫۸۲۰ / ۴	db.stats
هویت بین‌منبعی کار می‌کند	آگهی‌های شناسایی شده به‌عنوان یک خودرو در چند سایت	۱٫۰۵۸	canonical.find_duplicates
واژگان خودآموز است	برند / مدل استخراج شده از خود پیکره	۷۵۱ / ۱۹۳	lexicon.build
پوشش تست جدی است	تعداد تست جمع‌آوری شده	۲۲۴	backend/tests/

یافته قابل توجه: صداقت به جای عدد زیباتر

افزودن منابع سوم و چهارم، میانگین خطای مدل را بالا برد (از حدود ۱۲٪ به ۱۴٪). این پسرفت نیست: سه‌چهارم پیکره حالا از آگهی‌هایی تشکیل شده که به مراتب کمتر از باما درباره خودشان اطلاعات منتشر می‌کنند. به جای انتشار یک عدد ترکیبی با اطمینان یکسان، کاپوت اطمینان هر برآورد را متناسب با شواهد واقعی آن آگهی تنزیل می‌کند – میانگین اطمینان باما ۰/۵۹۵ و کارنامه ۰/۲۱۸ است.

۲ مسئله: سه پرسشی که هیچ سایتی پاسخ نمی‌دهد

خرید خودروی دست‌دوم در ایران یعنی پاسخ‌دادن به سه پرسش که هیچ‌کدام از سایت‌های آگهی موجود به آن‌ها پاسخ نمی‌دهند – و یک مسئله چهارم که هر سه را بدتر می‌کند.

۱. این قیمت منصفانه است؟

آگهی «قیمت پیشنهادی فروشنده» را نشان می‌دهد. هیچ‌چیز به خریدار نمی‌گوید این خودرو واقعاً چقدر می‌ارزد. مقایسه دستی با ده آگهی مشابه، کاری است که خریدار باید خودش انجام دهد.

۲. این خودرو سالم است؟

«دور رنگ»، «موتور تعویض شده»، «سند در رهن» – چیزهایی که واقعاً اهمیت دارند داخل متن توصیف دفن شده‌اند، اگر اصلاً ذکر شده باشند. هیچ فیلد ساختاریافته‌ای این‌ها را حمل نمی‌کند.

۳. به درد من می‌خورد؟

می‌توان بر اساس برند و سال فیلتر کرد. نمی‌توان گفت «اولین ماشین خانواده زیر ۸۰۰ میلیون که کم‌مصرف باشد». نیت خریدار در هیچ فیلتری نمی‌گنجد.

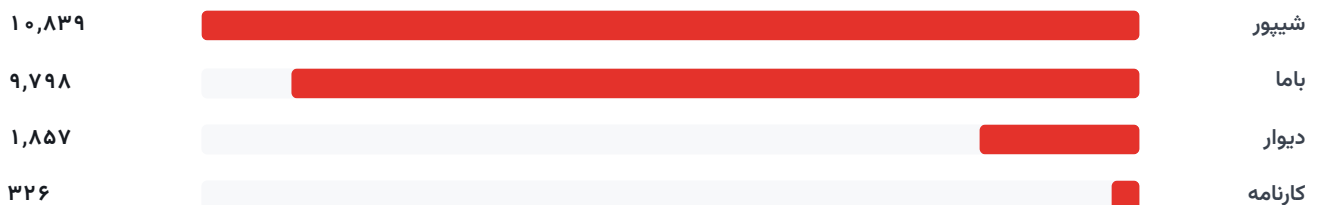
مسئله چهارم: آگهی‌هایی که اصلاً قیمت ندارند

نقطه کور بازار

از ۲۲,۸۲۰ خودروی دست‌دوم این پیکره، ۲,۶۲۱ آگهی هیچ قیمتی منتشر نمی‌کنند و فقط می‌نویسند «توافقی». در هر سایت موجودی این‌ها یک بن‌بست‌اند: در جست‌وجوی قیمتی نمی‌آیند، در فیلتر بودجه نمی‌گنجد و با هیچ خودروی دیگری قابل مقایسه نیستند. یعنی حدود ۱۱/۵٪ عرضه بازار برای خریداری که با بودجه جست‌وجو می‌کند، نامرئی است.

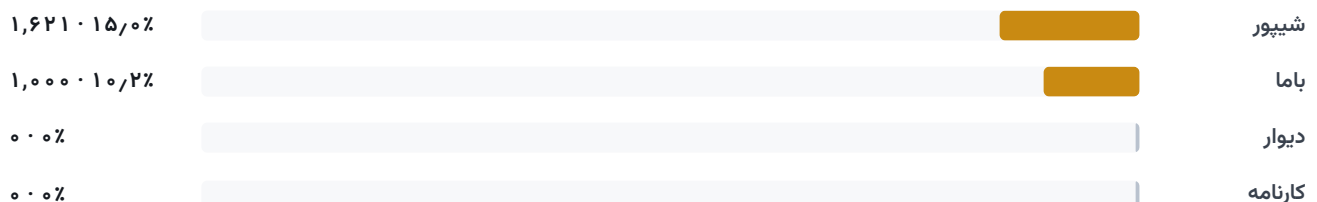
ترکیب پیکره بر اساس منبع

تعداد آگهی خودروی دست‌دوم پس از حذف خودروهای صفرکیلومتر – اندازه‌گیری از cars.db



سهم آگهی‌های «توافقی» از هر منبع

دیوار و کارنامه در این پیکره آگهی بدون قیمت ندارند؛ بار اصلی مسئله روی شیپور و باما است



طول میله نسبت به کل آگهی‌های همان منبع محاسبه شده است

چرا این مسئله فنی سختی است

قیمت‌گذاری خودروی بدون قیمت یک مسئله رگرسیون نظارت‌شده با یک پیچ ناخوشایند است: دقیقاً همان سطرهایی که بیشترین نیاز را به امتیازدهی دارند، برچسب ندارند. مدل باید روی ۱۹,۲۸۱ آگهی قیمت‌دار قابل‌آموزش یاد بگیرد و بعد روی سطرهایی تعمیم دهد که توزیعشان لزوماً یکسان نیست – فروشنده‌ای که قیمت را پنهان می‌کند، انگیزه‌ای دارد که ممکن است با قیمت خودرو همبسته باشد.

روی این، سه چالش داده‌ای دیگر سوار است که در بخش‌های ۷ و ۹ به تفصیل می‌آیند: تقویم دوگانه جلالی/میلادی در یک فیلد واحد، آگهی‌هایی که پیش‌پرداخت را به جای قیمت تبلیغ می‌کنند، و منابعی که هر کدام واژگان فارسی متفاوتی برای یک مفهوم یکسان دارند.

۳ راهکار کاپوت در یک نگاه

نام محصول، خود استدلال آن است. هر آگهی یک کاپوت بسته است – فروشنده رنگ بدنه را نشان می‌دهد و بقیه را نگه می‌دارد. کار محصول این است که کاپوت را باز کند.

قابلیت	چه می‌کند	چه چیزی آن را پشتیبانی می‌کند
قیمت منصفانه	هر آگهی یک برآورد بازار و یک نشان اختلاف می‌گیرد («۱۲٪ زیر قیمت بازار»)، همراه با بازه اطمینان و فهرست آگهی‌های قابل مقایسه‌ای که برآورد بر آن‌ها استوار است.	مدل <code>HistGradientBoosting</code> روی لگاریتم قیمت، به علاوه محاسبه عمق هم‌گروه واقعی
امتیاز سلامت	عددی ۰ تا ۱۰۰ که از وضعیت رنگ، تناسب کارکرد با سن، اعتبار فروشنده، کارشناسی، وضعیت شاسی و عبارات پیریسک متن آگهی ساخته می‌شود. هر امتیاز تفکیک‌شده نمایش داده می‌شود.	موتور قاعده‌محور قطعی + استخراج ریسک از متن
تناسب با نیاز	خریدار به فارسی ساده می‌نویسد؛ پرسش به نیت ساختاریافته تبدیل می‌شود و نتایج بر مبنای ارزش × سلامت × تناسب مرتب می‌شوند – با نمایش دلیل روی هر کارت.	تحلیل‌گر قاعده‌محور + لایه LLM اختیاری روی آن
فیلترهای مشتق‌شده	۱۹ ویژگی قابل فیلتر که فهرست مقادیرشان از خود پیکره ساخته می‌شود، با شمارش «یکی‌راکنارگذار» تا انتخاب چندگانه ممکن بماند.	<code>features.build_catalogue</code> و یک دروازه فیلتر مشترک
ارزش‌گذاری خودروی کاربر	کاربر خودروی خودش را توصیف می‌کند و همان موتوری که ۲۲,۸۲۰ آگهی را قیمت‌گذاری کرده، خودروی او را قیمت می‌زند و آگهی‌های زنده مشابه را نشان می‌دهد.	<code>appraise.py</code> – بازاستفاده کامل از اجزای موجود

اصل طراحی حاکم بر کل محصول

هیچ عددی بدون دلیل نمایش داده نمی‌شود. امتیاز سلامت به فاکتورهای برجسب‌دار با تأثیر عددی شکسته می‌شود؛ برآورد قیمت به آگهی‌های مقایسه‌ای واقعی که کاربر می‌تواند خودش بازرسی کند؛ و رتبه‌بندی به فهرست دلایلی که روی همان کارت چاپ می‌شود. یک رتبه‌بندی که کاربر نتواند بازخواستش کند، فقط یک نظر است.



۴ معماری فنی و نقشه مازول‌ها

معماری روی یک ایده استوار است: هر لایه فقط یک نوع داده را می‌شناسد. کراولر فقط رکورد خام می‌شناسد، آداپتور منبع فقط Listing تولید می‌کند، و همه چیز پایین‌دست – قیمت‌گذاری، سلامت، جست‌وجو – فقط Listing می‌بیند. نتیجه این است که افزودن یک سایت جدید هرگز به منطق محصول دست نمی‌زند.



نقشه ماژول‌های بک‌اند

ماژول	خط	مسئولیت
crawl/base.py	۱۵۱	قرارداد Source ، رجیستری آداپتورها، لایه درخواست با تلاش مجدد و throttle تطبیقی
crawl/bama.py	۱۶۸	کلاینت API باما؛ صفحه‌بندی با pageIndex ، کرال لایه‌بندی‌شده بر اساس برند، نقطه پایانی جزئیات
crawl/sheypoor.py	۳۰۹	آداپتور شیپور؛ مشخصات کامل داخل پاسخ جست‌وجو، تأیید دسته از بردکرامب هر سطر
crawl/divar.py	۴۲۰	آداپتور دیوار؛ پیمایش ویجت‌های تودرتو، استنتاج وضعیت بدنه از متن، استخر کارگر برای جزئیات
crawl/karnameh.py	۲۲۵	آداپتور کارنامه؛ استخراج buildId نکست‌جی‌اس، رکوردهای تایپ‌شده
crawl/run.py	۱۷۱	خط فرمان کرال؛ ازسرگیری با --append ، جداسازی منابع، ادامه در صورت خرابی یک منبع
normalize.py	۴۳۲	ساختار Listing و همه تبدیل‌های خالص و قابل تست
canonical.py	۲۲۱	BrandResolver و تشخیص محافظه‌کارانه آگهی تکراری
ingest.py	۱۵۸	خواندن JSONL هر منبع، تفکیک، حل هویت، حذف صفرکیلومترها، نوشتن در SQLite
db.py	۳۱۵	اسکیما، مهاجرت ستون، مسیر خواندن واحد (_FULL_SELECT)، هرس سطرهای کهنه
pricing.py	۴۱۸	آموزش، امتیازدهی، تشخیص پیش‌پرداخت، محاسبه اطمینان
health.py	۲۷۳	۹ فاکتور امتیاز سلامت، هر کدام با برچسب دوزبانه و تأثیر عددی
enrich.py	۲۲۸	اسکنر کلیدواژه‌ای + ادغام خروجی LLM؛ کار دسته‌ای، نه در مسیر درخواست
llm.py	۱۴۴	کلاینت سازگار با OpenAI، کش دیسکی، هرگز خطا به بالا پرتاب نمی‌کند
query.py	۳۶۶	تحلیل‌گر فارسی قاعده‌محور + ادغام LLM، در قالب Intent
lexicon.py	۴۵۲	واژگان برند/مدل مشتق‌شده، تاشدگی املایی فارسی، تطبیق موجودیت
search.py	۵۹۳	نمایه TF-IDF دوگانه، LSA کش‌شده، دو دروازه صحت پرسش
features.py	۸۶۱	کاتالوگ ۱۹ ویژگی، effective_price ، شمارش یکی‌راکنارگذار
rank.py	۴۵۷	وزن‌دهی پویا، جداسازی «مرتب‌سازی» از «توضیح‌دادن»
appraise.py	۶۰۴	ارزش‌گذاری خودروی خارج از پیکره، به‌علاوه منطق امتناع صادقانه
main.py	۵۶۴	اپلیکیشن FastAPI، بارگذاری پیکره در حافظه، کش جست‌وجو

مجموع: حدود ۷,۶۰۰ خط پایتون در بک‌اند و ۵,۲۰۰ خط TypeScript/TSX در فرانت‌اند، به‌علاوه ۲,۱۰۰ خط تست و ارزیابی.

۵ لایه کراول: چهار منبع، یک قرارداد

داده از نقطه‌های پایانی JSON عمومی می‌آید، نه از خراش HTML – بدون مرورگر بی‌سر و بدون سلکتور شکننده. هر سایت یک آداپتور Source است که فقط دو چیز را مالک است: گرفتن رکوردهای خام، و تبدیل یکی از آن‌ها به Listing.

مقایسه چهار منبع

منبع	نقطه پایانی	آگهی	درخواست به ازای خودرو	ویژگی متمایز
باما غنی‌ترین	GET /cad/api/search?pageIndex=N&brand=... GET /cad/api/detail/{code}	۹,۷۹۸	۱/۳۰~	تنها منبعی که حجم موتور، قدرت، مصرف سوخت و امتیاز نمایشگاه منتشر می‌کند؛ برند لاتین در کنار فارسی
شیپور ارزان‌ترین	GET /api/v10.0.0/search?c=43627&p=N	۱۰,۸۳۹	۱/۲۴	کل مشخصات داخل پاسخ جست‌وجو؛ وضعیت بدنه را اعلام می‌کند؛ تنها منبع دارای وضعیت شاسی
دیوار گران‌ترین	POST /v8/postlist/w/search GET /v8/posts-v2/web/{token}	۱,۸۵۷	۱~	مهل‌ت بیمه شخص ثالث و وضعیت معاینه فنی – که باما ندارد
کارنامه تمیزترین	GET /_next/data/{buildId}/buy-used-cars.json	۳۲۶	۱/۲۰	اعداد صحیح تایپ‌شده به‌جای رشته نمایشی؛ brand_name_fa کنار brand_name_en پرچم کارشناسی

هزینه کراول: درخواست HTTP به ازای هر خودرو

شیپور با یک درخواست ۲۴ خودروی کاملاً مشخص‌شده می‌دهد؛ دیوار برای هر خودرو یک درخواست جزئیات لازم دارد

~۱/۰	دیوار
~۰/۰۳۳	باما
۰/۰۵۰	کارنامه
۰/۰۴۲	شیپور

سرعت مشاهده‌شده: شیپور حدود ۱۲ خودرو در ثانیه، دیوار حدود ۰/۷ خودرو در ثانیه

قرارداد آداپتور

```
# backend/app/crawl/base.py – کل سطح تماسی که یک منبع جدید باید پیاده کند
class Source(ABC):
    name: str
    delay: float # ادب، نه کارایی

    async def iter_raw(self, *, max_pages, **options) -> AsyncIterator[dict]:
        """رکوردهای خام را دقیقاً همان‌طور که سایت برگرداند تولید کن"""

    def normalize(self, raw: dict) -> Listing | None:
        """اگر غیرقابل استفاده بود None تبدیل کن، یا Listing یک رکورد خام را به"""
```

تصمیم: رکورد خام همیشه دست‌نخورده ذخیره می‌شود

خروجی کراول عیناً در `data/{source}_raw.jsonl` نوشته می‌شود. تمام تفکیک در آداپتور و `normalize.py` اتفاق می‌افتد. نتیجه: اصلاح یک باگ در نرمال‌سازی هرگز به معنای کراول دوباره نیست – و برای دیوار که یک پیمایش کامل ساعت‌ها طول می‌کشد، این تفاوت بین یک دقیقه و یک بعدازظهر است.

۵/۱ نحوه کراول هر سایت، به تفکیک

باما – صفحه‌بندی ساده، لایه‌بندی هوشمند

باما یک API غیر احراز هویتی دارد که خود اپلیکیشن وبش را تغذیه می‌کند. صفحه‌بندی فقط با `pageIndex` انجام می‌شود. پاسخ یک `cursor` مبهم هم حمل می‌کند (تنها روی اولین آگهی هر صفحه)، اما پس‌دادن آن نتیجه **بدتری** در حذف تکراری‌ها می‌دهد، پس استفاده نمی‌شود. مقدار `total_count` در متادیتا «مجموع تا اینجا» است نه اندازه پیکره، بنابراین توقف بر مبنای صفحه خالی انجام می‌شود نه بر مبنای آن عدد.

چرا کراول باما بر اساس برند لایه‌بندی می‌شود

خوراک بدون تفکیک به سمت «آنچه اخیراً منتشر شده» متمایل است. کراول جداگانه ۲۵ برند (`BAMA_BRANDS`) هم‌گروه‌های عمیق‌تری در سطح مدل می‌سازد – و عمق هم‌گروه دقیقاً همان چیزی است که اطمینان برآورد قیمت را تعیین می‌کند. یک شمارنده «بی‌ثمر» (۹۰ آگهی تکراری پشت سر هم) کراول هر برند را متوقف می‌کند تا زمان هدر نرود.

خوراک جست‌وجوی باما با اسلات‌های تبلیغاتی (`type == "banner"`) درهم است که در همان لایه آداپتور حذف می‌شوند. دو فیلد `trim_en` و `model` در خوراک جست‌وجو **صفر درصد پر شده‌اند** – فقط روی نقطه پایانی جزئیات وجود دارند. هر دو از اسلاگ URL بازبایی می‌شوند (بخش V).

شیپور – ارزان‌ترین منبع، با یک تله خاموش

شیپور با JSON:API صحبت می‌کند و `fullAttributes` را داخل پاسخ جست‌وجو می‌فرستد؛ یعنی برخلاف دیوار هیچ درخواست جزئیاتی لازم نیست. یک فراخوان ۲۴ HTTP خودروی کاملاً مشخص شده.

تله: شناسه دسته اشتباه، خطا نمی‌دهد

فیلتر دسته `c=43627` است که از `/api/v10.0.0/search/filters/car` خوانده شده، نه حدس زده شده. اشتباه‌گرفتن آن **بی‌صدا** است: بدون آن، نقطه پایانی با کمال میل کل سایت را برمی‌گرداند – `meta.total` عدد ۷۳۹,۷۹۱ را در همه دسته‌ها نشان می‌دهد به جای ۶۴,۴۲۹ خودرو – و سطرها آپارتمان‌اند. راه‌حل: آداپتور دسته را از بردکرامب **خود هر سطر** تأیید می‌کند، نه از پرس‌وجویی که فرستاده.

یک صفحه، آگهی واقعی را با `paidEngagement`، `banner`، `nativeAd` و `catalogLink` مخلوط می‌کند. نوع `vip` خودش آگهی نیست بلکه یک کاروسل است که `items` آن آگهی‌اند، پس باز می‌شود نه اینکه رد شود. شمارنده «۲۵ صفحه بی‌ثمر پشت سر هم» پایان کراول را تعیین می‌کند – سخاوتمندانه، چون یک اجرای ازسرگرفته‌شده ابتدا روی همه چیزی که قبلاً دارد راه می‌رود.

دیوار – داده نمایشی، نه رکورد

دیوار داده **ارائه** برمی‌گرداند نه رکورد: قیمت و کارکرد به‌صورت رشته نمایشی با ارقام فارسی می‌آیند و فیلدهای مفید داخل ویجت‌های تایپ‌شده رابط کاربری زندگی می‌کنند. نمای فهرست هیچ سال، برند، مدل یا وضعیت بدنه‌ای ندارد، پس درخواست جزئیات اجباری است – و همین دیوار را گران‌ترین منبع به ازای هر آگهی می‌کند.

آداپتور کل بار جزئیات را یک بار به‌صورت بازگشتی می‌پیماید (`_widgets`) و همان فهرست تخت را به هر سه استخراج‌کننده می‌دهد، به‌جای اینکه سه بار بپیماید. سه نوع ویجت مهم‌اند:

ویجت	چه می‌دهد
GROUP_INFO_ROW	جفت‌های برچسب/مقدار: کارکرد، مدل (سال تولید)، قیمت پایه، گیربکس، نوع سوخت، رنگ، وضعیت بدنه
parent_items (بردکرامب)	وسایل نقلیه < خودرو < پژو < 206 < تیپ ۵ ← برند، مدل، تیپ
DESCRIPTION_ROW	بلندترین متن آزاد صفحه = توصیف فروشنده

دو نکته که به‌سختی پیدا شدند

یک: `pagination_data.last_post_date` باید RFC۳۳۳۹ با Z باشد؛ فرستادن "0" پاسخ ۴۰۰ HTTP می‌دهد.

دو: کراول فقط تهران باعث شد این منبع روی ۱,۱۱۷ آگهی بماند – هر رکورد خام `city_persian == 'تهران'` داشت. حالا ۱۲ شهر با شناسه‌های تأییدشده از `GET /v8/places/cities` پیمایش می‌شوند و هر شهر جداگانه لاگ می‌شود تا یک شناسه اشتباه به‌صورت «صفر» دیده شود، نه اینکه بی‌صدا پیکره را کوچک کند.

دیوار وضعیت بدنه را روی بیشتر آگهی‌ها حذف می‌کند (پوشش اندازه‌گیری‌شده: ۵۳٪) – و وضعیت بدنه قوی‌ترین ورودی امتیاز سلامت است. پس `infer_body_status` آن را از عنوان و توصیف استنتاج می‌کند، **بدرترین تطبیق اول**، تا فروشنده‌ای که «بدون رنگ» را کنار «دور رنگ» می‌نویسد نتواند نمره تمیز بگیرد.

کارنامه – بدون API، اما با داده تایپ‌شده

کارنامه REST API عمومی ندارد، اما یک اپلیکیشن Next.js با مسیریاب `pages` است؛ یعنی داده‌ای که صفحات خودش از آن ساخته می‌شوند به‌صورت JSON در `/_next/data/{buildId}/buy-used-cars.json` سرو می‌شود.

- `?page=1` یک استاب تغییرمسیر (`__N_REDIRECT`) برمی‌گرداند و هیچ آگهی – صفحه یک باید **بدون** رشته پرس‌وجو درخواست شود.
- `buildId` با هر استقرار عوض می‌شود، پس نمی‌توان آن را ثابت کرد؛ در ابتدای هر کراول از HTML صفحه استخراج می‌شود.
- اگر پیدا نشد، آداپتور **خطا پرتاب می‌کند و متوقف می‌شود** – عمداً. یک کراول که با صفر رکورد موفق شود، به‌مراتب گران‌تر از یک کرش است، چون کسی متوجهش نمی‌شود.

چرا کارنامه با ۳۲۶ آگهی هنوز ارزشش را دارد

کارنامه برای کیفیت داده اینجاست، نه حجم. سه چیز می‌دهد که هیچ منبع دیگری نمی‌دهد: اعداد صحیح تایپ‌شده به‌جای رشته نمایشی؛ `brand_name_en` در کنار `brand_name_fa` که نگاشت هویت بین‌منبعی را تقویت می‌کند (تا پیش از آن فقط باما این کار را می‌کرد)؛ و پرچم `has_inspection`.

یک فیلد که فریبنده بود

`prepayment_amount` شبیه پاسخ به مسئله «پیش‌پرداخت به‌جای قیمت» به نظر می‌رسید. نبود: اندازه‌گیری روی ۶۰ آگهی نشان داد روی همه آن‌ها حاضر است و بین ۶۳ تا ۷۹ درصد `price` می‌ایستد، فارغ از اینکه `is_leasing_available` ست شده باشد یا نه. یعنی یک پیشنهاد تأمین مالی است که کارنامه برای هر خودرو محاسبه می‌کند، نه چیزی که فروشنده تبلیغ کرده. پس عمداً ثبت نمی‌شود.

سایت‌هایی که امتحان شدند و کار نکردند

سایت	چرا خارج ماند
همراه مکانیک	مسیر <code>/car-buy/</code> یک صفحه سرویس کانسیرج است که <code>__NEXT_DATA__</code> آن فقط متاتگ حمل می‌کند
خودرو ۴۵	داده <code>/used-car/</code> React flight می‌فرستد بدون هیچ کلید آگهی؛ زیردامنه <code>buy.</code> به هیچ API ای ارجاع نمی‌دهد
ایران‌کار	یک SPA با Nuxt که مسیرهای آگهی‌اش روی <code>fetch</code> مستقیم ۴۰۴ می‌دهند

هر سه دوباره بررسی شدند، نه اینکه فرض شود. هر کدام به مهندسی معکوس مبتنی بر مرورگر و یک تفکیک‌کننده اختصاصی و شکننده نیاز داشتند — که با اصل «بدون مرورگر بی‌سر، بدون سلکتور شکننده» در تضاد است.

۶ کنترل نرخ تطبیقی و اخلاق کراول

مهم‌ترین درس این لایه یک جمله است: **توان عبوری، خاصیت سایت است، نه خاصیت تعداد کارگرهای ما.** کراولر به جای اینکه سرعتش را به او بگویند، سرعت هر سایت را خودش پیدا می‌کند.

آزمایشی که شکست خورد

اولین تلاش برای سریع‌تر کردن دیوار، یک استخر پنج کارگره همزمان بود. اندازه‌گیری شده: $1/2$ برابر سریع‌تر و 8 آگهی از 48 گم‌شده. دلیل این است که دیوار انفجار را محدود می‌کند نه توان عبوری را: نقطه پایانی جزئیاتش در حدود 70 میلی‌ثانیه پاسخ می‌دهد، حدود سی درخواست را سرو می‌کند و بعد تا پرشدن دوباره سهمیه، 429 برمی‌گرداند. با پنج کارگر نیمی از درخواست‌ها 429 می‌گیرند و با ده کارگر همه‌شان.

تلاش مجدد ثابت سه‌باره روی چنین درخواستی فقط آگهی را به تلاش مجدد تبدیل می‌کند — و بی‌صدا، چون یک جزئیات ازدست‌رفته دقیقاً شبیه خودرویی است که در تفکیک شکست خورده.

راه‌حل: 429 کل منبع را کند می‌کند

```
# backend/app/crawl/base.py – Source._request
if resp.status_code == 200:
    # در دو-سه موفقیت، مستقیم به سقف برمی‌گردد throttle و پاشی آهسته. ریختن.
    self._throttle = self._throttle * 0.92 if self._throttle > 0.05 else 0.0
    return resp.json()

if resp.status_code == 429:
    # نایب MAX_THROTTLE_S = 8.0 دو برابر شدن، با سقف
    self._throttle = min(MAX_THROTTLE_S, max(1.0, self._throttle * 2))
    continue
```

اثر اندازه‌گیری شده روی نرخ ازدست‌رفتن آگهی (دیوار، چهار صفحه)

throttle تطبیقی + کاهش همزمانی از 5 به 2

۴۲٪		قبل – گم‌شده
۱۰٪		بعد – گم‌شده

پارامترهای ادب، به تفکیک منبع

منبع	تأخیر بین درخواست	همزمانی	یادداشت
باما – جست‌وجو	$1/0$ ثانیه	۱	ادب، نه کارایی
باما – جزئیات	$0/4$ ثانیه	۱	ترتیبی و عمدی؛ چند صد غنی‌سازی کافی است
شیپور	$1/2$ ثانیه	۱	اندازه‌گیری شده: حلقه تنگ‌تر از این حدود صفحه 2 تایم‌اوت می‌دهد
دیوار	$0/35$ ثانیه	۲	تأخیر هر کارگر برای خودش، نه مکث سراسری
کارنامه	$1/0$ ثانیه	۱	18 صفحه؛ حجم اصلاً مسئله نیست

شناسه‌های موجود روی دیسک قبل از شروع خوانده می‌شوند و به آداپتور پاس داده می‌شوند، نه اینکه بعد از دریافت فیلتر شوند – تفاوت این دو برای دیوار یعنی صرفه‌جویی در یک درخواست جزئیات به ازای هر آگهی تکراری، که کل هزینه آن منبع است.

ملاحظات اخلاقی و حقوقی

همه داده‌ها از نقطه‌های پایانی JSON عمومی و بدون احراز هویت می‌آیند، یک‌بار و با حجم متعارف، با تأخیر بین درخواست‌ها و کراولری که وقتی سایت می‌گوید «آهسته‌تر» خودش را کند می‌کند. این یک دموی غیرتجاری است؛ تمام داده آگهی‌ها و تصاویر متعلق به همان سایت‌ها و فروشندگان اصلی می‌ماند.

۷ نرمال سازی: از رشته نمایشی تا رکورد تایپ شده

`normalize.py` وجود دارد چون داده منبع واقعاً به هم ریخته است – و به شکل هایی که برای قیمت گذاری اهمیت دارند. همه چیز در این مازول خالص و بدون اثر جانبی است تا بتوان واحد آزمایی کرد و بدون کراول دوباره روی JSONL خام اجرا کرد.

هفت چالش نرمال سازی و پاسخ هر کدام

چالش	شکل مسئله در داده خام	راه حل
تقویم دوگانه	سال مدل برای خودروی داخلی جلالی است (1391) و برای وارداتی میلادی (2012) – در یک فیلد واحد و بدون هیچ پرچمی	آستانه <code>JALALI_MAX = 1500</code> ؛ سال های جلالی پیکره ۱۳۷۰ تا ۱۴۰۵ و میلادی ۲۰۰۰ تا ۲۰۲۶ هستند، پس شکاف بدون ابهام است. افست ۶۲۱ سال اعمال می شود و <code>year_display</code> نگه داشته می شود تا رابط کاربری همان سالی را نشان دهد که فروشنده تبلیغ کرده
کارکرد	یا رشته «صفر کیلومتر» است یا «۲۴۰,۰۰۰ km»	فقط توکن «صفر» نشانگر قابل اتکای کارکرد صفر است. هرگز زیررشته ای مثل <code>0 km</code> تطبیق داده نمی شود – چون <code>240,000 km</code> شامل <code>0 km</code> است
ارقام	فارسی (۱۲) یا عربی هندی (۱۲) یا لاتین، گاهی در یک رشته	جدول ترجمه یکپارچه <code>fa_to_en_digits</code> که هر دو مجموعه را به ASCII می برد، پیش از هر تفکیک عددی
قیمت توافقی	باما آن را <code>type='negotiable'</code> با قیمت <code>'0'</code> کد می کند؛ شیپور رشته «توافقی» داخل <code>amount</code> می گذارد	حیاتی نتیجه <code>NULL</code> است، هرگز <code>0</code> – یک صفر هدف رگرسیون را مسموم می کرد، و این سطرها دقیقاً همان هایی اند که محصول برای خدمت به آنها وجود دارد
وضعیت بدنه	یک عبارت فارسی روی مقیاس شدت ضمنی، با واژگان متفاوت در هر سایت	نگاشت به مقیاس ۰ تا ۱۰۰ «یکپارچگی رنگ». هر منبع جدول عبارت خودش را دارد؛ عبارت ناشناخته به تطبیق زیررشته ای و در نهایت به پیش فرض میانه ۶۰ می افتد
مدل و تیپ غایب	خوراک جست و جوی باما <code>model</code> و <code>trim_en</code> را اصلاً ندارد (۰٪ پرشدگی)	<code>parse_url_slug</code> هر دو را از اسلاگ URL بازیابی می کند – بدون آن هر خودرو یک کلید هم گروه خالی مشترک داشت و مقایسه ها بی معنا بودند
گیربکس و سوخت	«دنده ای» / «دنده ای»، «دوگانه سوز» / «دوگانه سوز»، «گازوئیل» / «دیزل»	جدول های مشترک <code>TRANSMISSION_MAP</code> و <code>FUEL_MAP</code> در <code>normalize.py</code> ، که هر سه آداپتور از آنها می خوانند

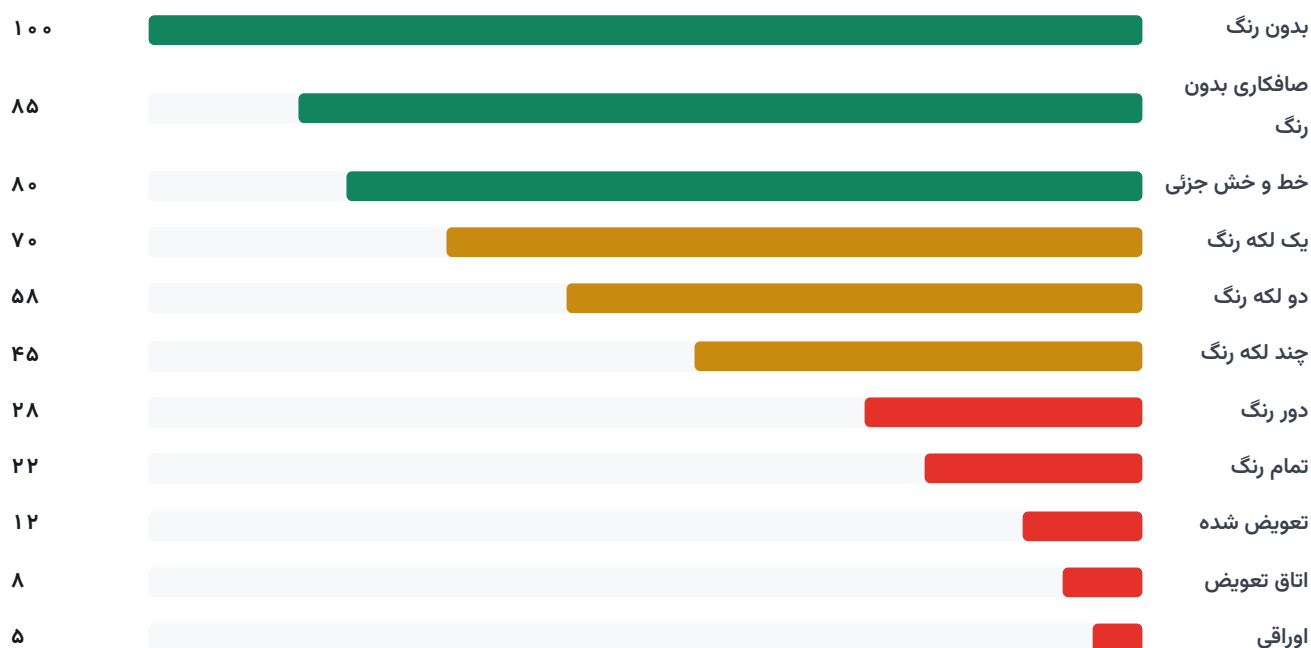
بازیابی مدل و تیپ از اسلاگ URL

```
/car/detail-tocysn01-mazda-3sedan-2.0ltype4-1395
└──code└──brand└──model└──trim└──year
```

الگوریتم، بخش هایی را که از قبل می دانیم (کد، برند، سال انتهایی) پوسته برداری می کند و باقی را می شکافد – که در برابر مدل ها و تیپ های چندتوکنی مقاوم است. نتیجه اندازه گیری شده روی پیکره فعلی: ۱,۰۶۶ هم گروه سطح تیپ با پنج مقایسه ای قیمت دار یا بیشتر.

نگاشت عبارت فارسی وضعیت بدنه به امتیاز ۰ تا ۱۰۰

ترتیب از روی نحوه واقعی تنزیل قیمت در بازار ایران - نه از روی حدس



استنتاج وضعیت بدنه از متن: بدترین تطبیق اول

وقتی منبع فیلد را اعلام نمی‌کند (دیوار، ۴۷٪ آگهی‌ها)، عبارت از عنوان و توصیف استخراج می‌شود. الگوها از بدترین به بهترین مرتب شده‌اند تا جمله‌ای مثل «بدون رنگ به جز دور رنگ گلگیر» نتواند به عنوان خودروی تمیز خوانده شود. عبارت‌بندی خوش‌بینانه فروشنده نباید امتیاز را باد کند.

۸ هویت بین‌منبعی و تشخیص آگهی تکراری

باما و دیوار یک پژو ۲۰۶ واحد را کاملاً متفاوت توصیف می‌کنند. بدون یک نگاشت مشترک، هر خودروی دیوار در هم‌گروه تنهای خودش می‌افتد، مدل قیمت نمی‌تواند مقایسه‌ای‌ها را بین منابع جمع کند، و یک خودروی واحد که دو بار آگهی شده، دو خودروی متفاوت به نظر می‌رسد.

منبع	brand	model	آنچه واقعاً می‌دهد
باما	peugeot	206ir	عنوان پژو، 206 – هر دو نیم منتشر می‌شود
کارنامه	peugeot	206	brand_name_en کنار brand_name_fa
دیوار	تهی	تهی	بردکرامب ['پژو', '206', 'تیپ ۵'] – فقط فارسی
شیپور	تهی	تهی	بردکرامب فارسی + «مدل خودرو» در attributes

نگاشت آموخته می‌شود، نه دست‌نویس

نام فارسی کلید اتصال است، چون تنها چیزی است که هر چهار سایت روی آن توافق دارند. نگاشت از باما و کارنامه آموخته می‌شود – دو منبعی که brand و brand_fa را با هم منتشر می‌کنند – پس تقریباً هیچ‌چیز دست‌نویس نیست. برای نام مدل که ممکن است به چند اسلاگ نگاشت شود («سراتو» ← cerato یا ceratoir)، رأی‌گیری فراوانی انجام می‌شود تا هم‌گروه‌ها پایدار بمانند.

_model_candidates شکل‌های قابل امتحان یک نام فارسی را تولید می‌کند، چون سایت‌ها بر سر اینکه برند بخشی از نام مدل هست یا نه توافق ندارند: شیپور «دنا پلاس» و «پیکان وانت» می‌نویسد جایی که باما فقط «پلاس» و «وانت» می‌گوید. حذف برند پیشوندی این‌ها را بدون نوشتن حتی یک نگاشت دستی بازیابی می‌کند.

باگ واقعی: «رانا» که ناپیدا شد

باما اصلاً رانا ندارد، پس تا پیش از ورود کارنامه هیچ‌چیز این مارک را یاد نمی‌داد. کارنامه آن را Runna می‌نویسد. نتیجه: ۲۶ رانا زیر اسلاگ runna بایگانی شدند در حالی که پرسش کاربر به rana تبدیل می‌شد، و جست‌وجو روی همه آن‌ها پاسخ داد «چنین خودرویی نداریم».

اصلاح ساختاری بود، نه املائی: واژگان برندی که جست‌وجو استفاده می‌کند حالا خود BrandResolver را بذرپاشی می‌کند، پس یک اسلاگ آموخته از کراول دیگر نمی‌تواند با اسلاگی که پرسش خریدار به آن حل می‌شود در تضاد باشد.

تشخیص تکراری – عمداً محافظه‌کارانه

یک ادغام اشتباه یک آگهی واقعی را پنهان می‌کند، که بدتر از نشان‌دادن دوباره یک خودرو است. پس شرط‌ها سخت‌گیرانه‌اند:

- همان سطل برند / مدل / سال
- کارکرد با اختلاف حداکثر ۲٪ و قیمت با اختلاف حداکثر ۳٪
- دو منبع متفاوت – دو آگهی روی یک سایت هرگز ادغام نمی‌شوند
- آگهی بدون کارکرد اصلاً وارد سطل نمی‌شود، چون هرگز نمی‌تواند شرط نزدیکی را برآورده کند

بهینه‌سازی الگوریتمی

هر منبع اضافه، سطل‌ها را متراکم‌تر می‌کند و تعداد جفت‌ها با مربع آن رشد می‌کند. مرتب‌کردن هر سطل بر اساس کارکرد اجازه می‌دهد حلقه زودتر بشکند: به محض اینکه فاصله از عضو چپ بیش از آستانه شود، برای سطرهای بعدی فقط بیشتر می‌شود. نتیجه اندازه‌گیری شده: ۱,۰۵۸ آگهی که روی بیش از یک سایت فهرست شده‌اند شناسایی و به هم پیوند خورده‌اند.

پوشش فیلد به تفکیک منبع – چرا هویت مشترک لازم است

درصد آگهی‌های هر منبع که یک فیلد را دارند

اندازه‌گیری مستقیم از cars.db – این جدول توضیح می‌دهد چرا خطای مدل بین منابع این قدر فرق می‌کند

فیلد	باما	شیپور	دیوار	کارنامه	کل پیکره
حجم موتور	۹۲%	۰%	۰%	۰%	۳۹%
مصرف سوخت	۸۹%	۰%	۰%	۰%	۳۸%
امتیاز نمایشگاه	۱۶%	۰%	۰%	۰%	۷%
قدرت موتور	۳%	۰%	۰%	۰%	۱%
نوع بدنه	۱۰۰%	۰%	۰%	۰%	۴۳%
وضعیت بدنه	۱۰۰%	۹۹%	۵۳%	۰%	۹۴%
وضعیت شاسی	۰%	۷۹%	۰%	۰%	۳۷%
بیمه شخص ثالث	۰%	۰%	۸۵%	۰%	۷%
توصیف متنی	۸۸%	۱۰۰%	۱۰۰%	۱۰۰%	۹۵%
کارکرد	۱۰۰%	۹۵%	۱۰۰%	۱۰۰%	۹۸%
سال تولید	۱۰۰%	۹۹%	۱۰۰%	۱۰۰%	۱۰۰%
گیربکس	۱۰۰%	۹۹%	۹۶%	۷۹%	۹۹%

پوشش بالا ■ متوسط رو به بالا ■ متوسط ■ کم ■ غایب

یافته: افزودن منبع، پوشش وضعیت بدنه را از ۶۶٪ به ۹۴٪ رساند

شیپور وضعیت بدنه را روی عملاً همه آگهی‌ها اعلام می‌کند، جایی که دیوار آن را حذف می‌کند. این یک برد واقعی برای موتور سلامت است. اما – و این نکته غیرمنتظره بود – برد قیمت‌گذاری نبود: خطای شیپور روی ۱۷/۴٪ می‌ایستد در برابر ۱۸/۵٪ دیوار. وضعیت رنگ اعلام‌شده برای امتیاز سلامت مهم است و برای قیمت، که به چگالی مشخصات فنی تکیه دارد، تقریباً بی‌اثر.

۹ موتور قیمت منصفانه

ادعای اصلی محصول این است که می‌توانیم به خریدار بگوییم یک خودرو واقعاً چقدر می‌ارزد – از جمله برای آگهی‌هایی که فروشنده قیمت را پنهان کرده. این یک مسئله رگرسیون نظارت‌شده با یک پیچ است: **سطرهایی که بیش از همه می‌خواهیم امتیاز بدهیم، دقیقاً همان‌هایی‌اند که برچسب ندارند.**

معماری مدل



ویژگی‌های ورودی



چرا source یک ویژگی مدل است

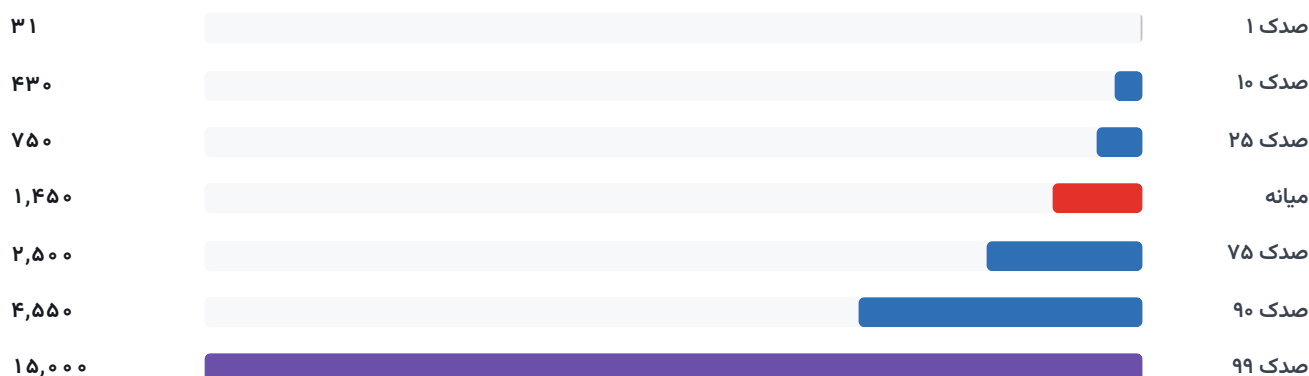
پلتفرم‌ها به‌طور سیستماتیک با هم فرق دارند – دیوار به فروشنده شخصی متمایل است و فیلدهای مشخصات کمتری حمل می‌کند. اگر مدل نبیند آگهی از کجا آمده، این تفاوت به **خودرو** نسبت داده می‌شود نه به پلتفرم. دادن این ستون به مدل، همان سوگیری را از برآورد خودرو جدا می‌کند.

چرا لگاریتم قیمت

قیمت خودرو در ایران سه مرتبه بزرگی را می‌پیماید. صدک ۱۰ پیکره حدود ۴۳۰ میلیون و صدک ۹۰ حدود ۴/۵۵ میلیارد تومان است، با دنباله‌ای تا ۴۵ میلیارد. بدون تبدیل لگاریتمی، خطای مطلق روی یک بنز خطای هر پراید در پیکره را غرق می‌کرد. در فضای لگاریتمی، خطای نسبی است که در کل دامنه قابل مقایسه می‌ماند.

توزیع قیمت منتشرشده در پیکره

میلیون تومان — بر مبنای ۲۰,۱۹۹ آگهی قیمت‌دار



دو یافته کیفیت داده که مدل را ماهیتاً تغییر دادند

یافته یک: آگهی‌هایی که پیش‌پرداخت را قیمت جا می‌زنند

بعضی نمایشگاه‌ها پیش‌پرداخت را به عنوان قیمت تبلیغ می‌کنند — دناهای نو با قیمت ۵۰ میلیون تومان در برابر ارزش بازار حدود ۲/۱ میلیارد. این‌ها آگهی‌های معمولی `lumpsum` هستند، پس هیچ فیلدی آن‌ها را شناسایی نمی‌کند — فقط به عنوان پرت شدید نسبت به هم‌گروه خودشان قابل تشخیص‌اند. حذف آن‌ها MAPE را از ۲۸٪ به ۱۱/۸٪ رساند.

یافته دو: حواله و پیش‌فروش، در زمان سرو هم باید مدیریت شوند

آگهی‌های حواله / پیش‌فروش / ثبت‌نام یک ادعا روی خودرو می‌فروشند، نه خودرو. عدد تبلیغ‌شده یک ودیعه است. اگر فقط در زمان آموزش حذف شوند، در زمان سرو هر رتبه‌بندی را به عنوان «۹۰٪ زیر قیمت بازار» تسخیر می‌کنند. حالا `price_flag` آن‌ها را `deposit` علامت می‌زند و هیچ اختلاف قیمتی برایشان محاسبه نمی‌شود. تعداد فعلی: ۸۲۴ آگهی.

یک ناسازگاری که مرتب‌سازی افشا کرد

مرتب‌سازی بر اساس قیمت، یک پورشه کاین ۱,۰۰۰ تومانی را اول گذاشت. بررسی پرت نسبت به هم‌گروه فقط وقتی کار می‌کند که هم‌گروه سه عضو یا بیشتر داشته باشد؛ کاین در هم‌گروهی از یک عضو نشسته بود. حالا `effective_price` به هر عدد منتشرشده زیر ۲۰ میلیون تومان، یا زیر ۱۵٪ برآورد خود آن خودرو، بی‌اعتماد است و به برآورد برمی‌گردد. این کار ۲۰ آگهی را بازطبقه‌بندی کرد که همه‌شان قیمت جانگهدار بودند (لندکروز ۱۱۰ هزار تومانی، کمری ۱۰ هزار تومانی). خودروهای واقعاً ارزان می‌مانند، چون آستانه عمدتاً به مراتب سخاوتمندانه‌تر از کف ۳۵٪ بررسی هم‌گروه است — یک برآورد، شاهد ضعیف‌تری از میانه یک هم‌گروه واقعی است و همان‌طور با آن رفتار می‌شود.

یک ماسک، دو پرسش

`price_flag` و `trainable_mask` زمانی با هم اختلاف داشتند: ماسک فقط عنوان را می‌خواند و پرچم، عنوان و توصیف را. نتیجه این بود که ۴۹۵ آگهی به عنوان قیمت بازار آموزش دیدند در حالی که همان ماژول آن عدد را ودیعه می‌نامید و از محاسبه اختلاف قیمت از آن امتناع می‌کرد. اصل حاکم: قیمتی که برنامه با بازار مقایسه نمی‌کند، قیمتی نیست که مدل باید از آن یاد بگیرد.

۹/۱ اطمینان: عددی که کسی نتواند بررسی کند، بدتر از نبود عدد است

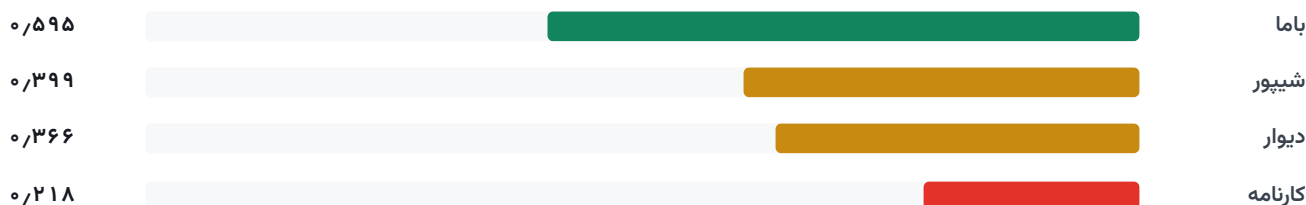
اطمینان از دو چیز ساخته می‌شود، و هر دو اندازه‌گیری شده‌اند نه سلیقه‌ای:

```
base          = 1 - exp(-n_comparables / 6.0)    # اشباعی، نه خطی
penalty        = {trim: 1.00, model: 0.82, global: 0.55}
evidence_factor = 0.6 + 0.4 * (آگهی واقعاً دارد)
confidence     = min(base * penalty * evidence_factor, 0.98)
```

- اشباعی بودن عمدی است. جهش از ۲ به ۸ مقایسه‌ای به مراتب بیشتر از ۴۰ به ۶۰ اهمیت دارد.
- هم‌گروه سطح مدل جریمه می‌شود چون تیپ‌های مختلف را قاطی می‌کند.
- کف ۰/۶ برای شواهد یعنی آگهی کم‌مشخصات تنزیل می‌شود، نه اینکه دور انداخته شود — برند، مدل، سال و کارکرد هنوز بیشتر سیگنال را حمل می‌کنند.

میانگین اطمینان برآورد، به تفکیک منبع

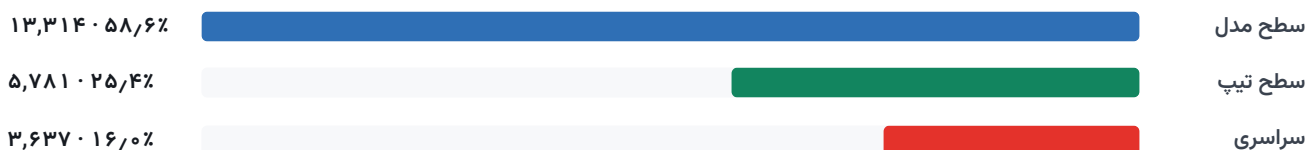
اندازه‌گیری از جدول pricing — این نمودار مستقیماً بازتاب چگالی مشخصات در نمودار پوشش فیلد است



میانگین کل پیکره: ۰/۴۷۸

سطح هم‌گروهی که هر برآورد روی آن ایستاده است

۲۲,۷۳۲ برآورد؛ آگهی به دلیل نداشتن سن قابل امتیازدهی نبودند



نتایج اندازه‌گیری شده روی داده کنارگذاشته

اعداد زیر با اجرای دوباره `pricing.train()` روی پیکره فعلی به دست آمده‌اند: تقسیم ۸۰/۲۰ با بذر ثابت ۴۲، آموزش روی ۱۵,۴۲۴ سطر و ارزیابی روی ۳,۸۵۷ سطر.

۰/۹۲۹

R^2 روی لگاریتم قیمت

۱۴/۴٪

MAPE

۷/۶٪

خطای میانه نسبی (median APE)

خطا به تفکیک منبع – چرا یک عدد ترکیبی گمراه‌کننده است

MAPE (میل به پرنس) و خطای میانه (عدد دوم) روی همان تقسیم کنار گذاشته



تعداد نمونه آزمون: باما ۱,۷۶۸ · شیپور ۱,۶۵۳ · دیوار ۳۷۳ · کارنامه ۶۳

چرا باما دو برابر آسان‌تر قیمت‌گذاری می‌شود

علت اندازه‌گیری شده است نه مرموز: باما تنها منبعی است که حجم موتور، قدرت، مصرف سوخت و امتیاز فروشنده را منتشر می‌کند و آن‌ها را روی حدود ۹۰٪ سطرها پر می‌کند، جایی که سه منبع دیگر تقریباً هیچ‌کدام را نمی‌دهند. پس اطمینان بر اساس **شواهدی که آگهی واقعاً حمل می‌کند** تنزیل می‌شود، نه فقط بر اساس تعداد مقایسه‌ای‌ها.

سنجه	اندازه‌گیری امروز	ثبت شده در README	توضیح اختلاف
MAPE	۱۴٫۴٪	۱۵٫۶٪	اجرای README پیش از اصلاحی انجام شده که <code>trainable_mask</code> را وادار کرد توصیف را هم مثل
خطای میانه	۷٫۶٪	۸٫۱٪	بخواند؛ آن اصلاح چند صد آگهی حواله را از مجموعه آموزش بیرون برد و نتیجه را کمی بهتر کرد. اندازه پیکره آموزش هم از ۱۹,۶۱۱ به ۱۹,۲۸۱ تغییر کرده است.
R ² (log)	۰٫۹۲۹	۰٫۹۱۷	

بازه قیمت: لنگر انداختن روی چیزی که مدل واقعاً اندازه می‌گیرد

نیمی از آگهی‌های کنار گذاشته در فاصله `median_ape` از قیمت واقعی خود می‌نشینند و میانگین توسط منابع کم‌داده تا `mape` کشیده می‌شود. پس یک برآورد خوش‌شاهد با خطای میانه نقل می‌شود و یک برآورد نحیف با میانگین، و بین این دو بر اساس اطمینان درون‌یابی می‌شود. جایگزین، یک درصد ثابت بود که به خودرویی با ۲۰۰ مقایسه‌ای و خودرویی با دو مقایسه‌ای **یک چیز** می‌گفت.

توزیع اختلاف قیمت آگهی نسبت به برآورد بازار

۱۹,۲۸۱ آگهی که هم قیمت منتشر شده دارند و هم برآورد – درصد منفی یعنی زیر قیمت بازار



میانه نزدیک صفر، شاهد بی‌طرفی مدل است: ۳,۹۷۸ آگهی بیش از ۸٪ زیر بازار و ۳,۳۹۵ آگهی بیش از ۱۲٪ بالای بازار

امتیاز سلامت عمده‌اً قاعده‌محور و کاملاً قطعی است. دو دلیل: سیگنال‌هایی که اهمیت دارند (وضعیت رنگ، تناسب کارکرد با سن، اعتبار فروشنده) از قبل ساختاریافته‌اند؛ و خریدار حق دارد دقیقاً بداند چرا یک خودرو این امتیاز را گرفت. مدل زبانی فقط شواهد اضافی از متن آزاد اضافه می‌کند – هرگز جای حساب را نمی‌گیرد.

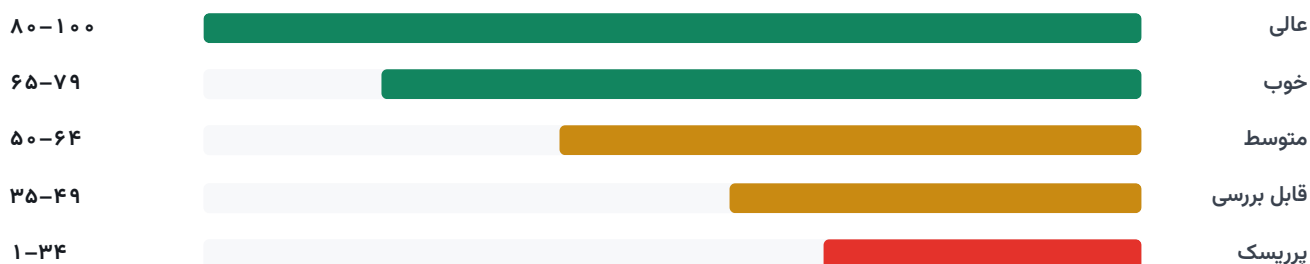
ده فاکتور و تأثیر عددی هرکدام

امتیاز پایه یک خودروی معمولی و بی‌ویژگی ۶۲ است. هر فاکتور از آنجا حرکت می‌کند:

فاکتور	شرط	تأثیر	منطق
وضعیت بدنه	خطی از امتیاز یکپارچگی رنگ: $(\text{grade} - 60) \times 0.40$	-۲۲ تا +۱۸	قوی‌ترین سیگنال وضعیت در این بازار؛ حداکثر دامنه نوسان بین همه فاکتورها
	وضعیت اعلام‌نشده	میان ۶۰	
کارکرد نسبت به سن	نسبت $\leq 1/8$ برابر انتظار	-۱۶	انتظار: ۲۰,۰۰۰ کیلومتر در سال. کارکرد غیرعادی کم روی خودروی قدیمی پاداش نمی‌گیرد بلکه ریسک ملایم است – در این بازار بیشتر نشانه دستکاری کیلومتر است تا خودروی پارکینگی
	نسبت $\leq 1/35$	-۹	
	نسبت $\geq 0/25$ و سن ≤ 5 سال	-۷	
	نسبت $\geq 0/65$ / متناسب	+۳ / +۱۰	
سن	≥ 1 سال / ≥ 5 سال	+۴ / +۸	فرسودگی و ایمنی، هر دو با سن همبسته‌اند
	≤ 12 سال / ≤ 18 سال	-۱۲ / -۶	
فروشنده	نماینده رسمی	+۸	پاسخ‌گویی قابل ردیابی. <code>dealer_ad_count</code> عمده‌اً خوانده نمی‌شود؛ پرکارترین نمایشگاه‌ها همان جالفا‌تده‌ترین‌ها هم هستند، پس شاهی در هیچ جهت نیست
	امتیاز نمایشگاه $\leq 4/3$ / $\leq 3/5$ / کمتر	+۷ / +۳ / -۶	
	فروشنده شخصی	-۲	
کارشناسی	پرچم کارشناسی‌شده منبع	+۱۲	باما و کارنامه این را می‌دهند؛ دیوار معادلش را با معاینه فنی معتبر می‌سازد
بیمه شخص ثالث	≤ 9 ماه / ≤ 4 ماه	+۶ / +۳	فقط دیوار در ایران دوسویه است: مهلت بلند پول واقعی است که خریدار خرج نمی‌کند، و بیمه منقضی هم هزینه فوری است هم نشانه اینکه خودرو مدتی خوابیده
	≤ 1 ماه / منقضی	-۲ / -۵	
وضعیت شاسی	شاسی تعویض‌شده	-۲۶	فقط شیپور آسیب سازه‌ای است نه ظاهری، پس بر هر امتیاز رنگی می‌چربد. تطبیق بدترین‌اول: «جلو سالم، عقب ضربه‌خورده» باید آسیب‌دیده امتیاز بگیرد نه سالم
	ضربه‌خورده / رنگ‌شده	-۲۰ / -۱۲	
	سالم و پلمپ / سالم	+۸ / +۵	
تصاویر	بدون تصویر	-۶	فروشنده‌ای که خودرو را پشت یک عکس پنهان می‌کند، بیشتر اوقات چیزی را پنهان می‌کند
	گالری ≤ 8 تصویر	+۳	
ریسک متنی	شدت بالا / متوسط / پایین	-۱۸ / -۱۰ / -۵	از <code>enrich.py</code> ؛ هر پرچم برچسب دوزبانه خودش را حمل می‌کند
	نکته مثبت (حداکثر دو مورد)	+۴ هرکدام	

تبدیل امتیاز عددی به برجسب قابل خواندن

هر نوار دوزبانه است تا رابط کاربری در هر دو زبان یک چیز بگوید



تصمیم: فیلد غایب یعنی «یک فاکتور کمتر»، نه جریمه

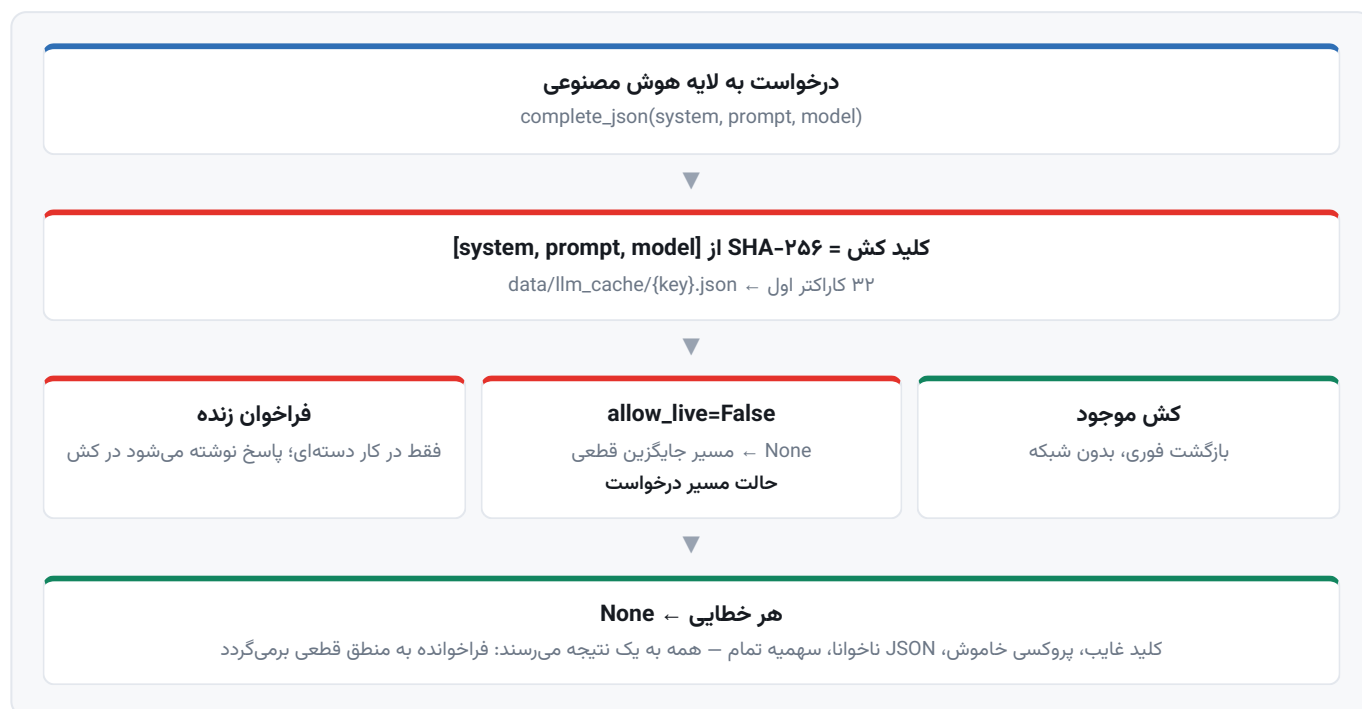
وضعیت شاسی فقط از شیپور می‌آید و بیمه فقط از دیوار. هیچ‌کدام برای منابعی که ساکت‌اند جریمه نمی‌سازند – صرفاً یک فاکتور کمتر در جمع. به همین دلیل هم وضعیت شاسی با پوشش ۳۷٪ در امتیاز سلامت حضور دارد اما از پنل فیلتر بیرون است: در فیلتر یک تله می‌شد که با یک کلیک دوسوم پیکره را حذف می‌کرد.

بهینه‌سازی: امتیاز کش‌شده در برابر امتیاز بازمحاسبه‌شده

مرتب‌کردن یک مجموعه نتیجه به امتیاز نیاز دارد اما به هیچ‌کدام از فاکتورهای پشت آن نه. `features.ensure_health_scores` عدد را در زمان بارگذاری روی هر سطر می‌نویسد، پس رتبه‌بندی هزاران کاندید به هزاران جست‌وجوی دیکشنری تبدیل می‌شود نه هزاران بار امتیازدهی مجدد. `score_listing` فقط برای ۲۴ کارتی که خواننده واقعاً می‌بیند اجرا می‌شود.

۱۱ هوش مصنوعی در کاپوت: سه کاربرد، یک قرارداد

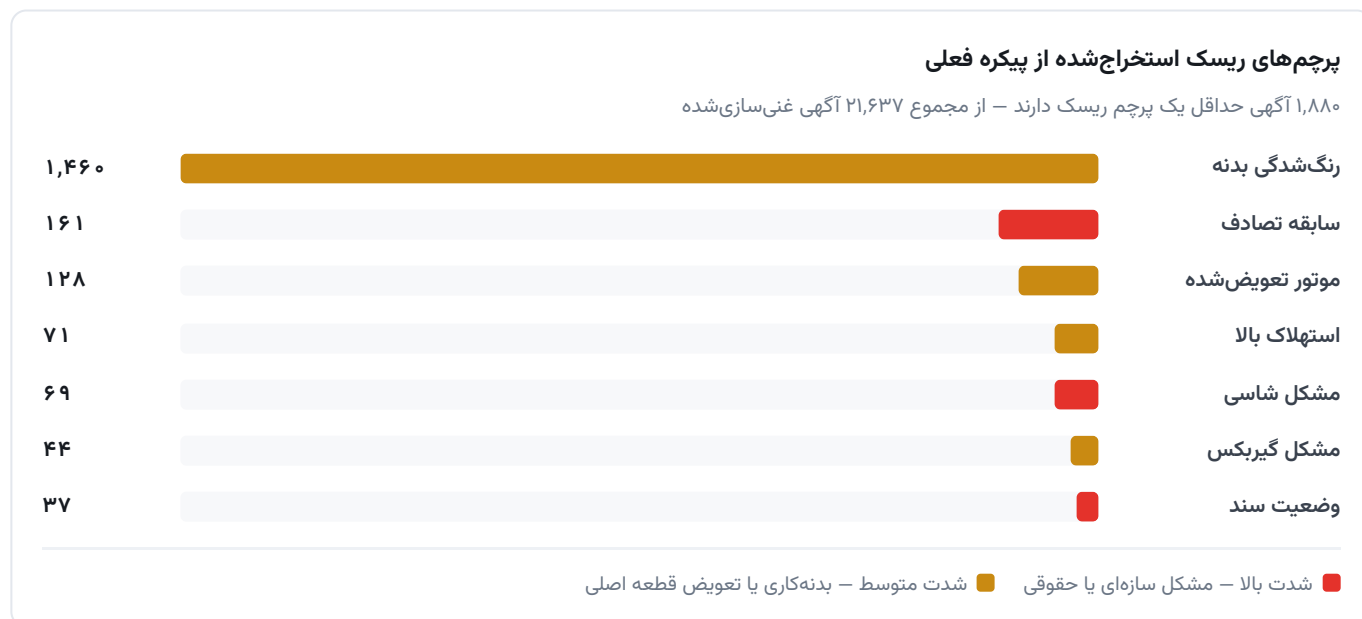
هوش مصنوعی در این پروژه سه جای مشخص دارد و هر سه پشت یک قرارداد واحد می‌نشینند: **اول کش، همیشه – و هرگز شکست درخواست.**



کاربرد یک: استخراج سیگنال ریسک از متن آگهی

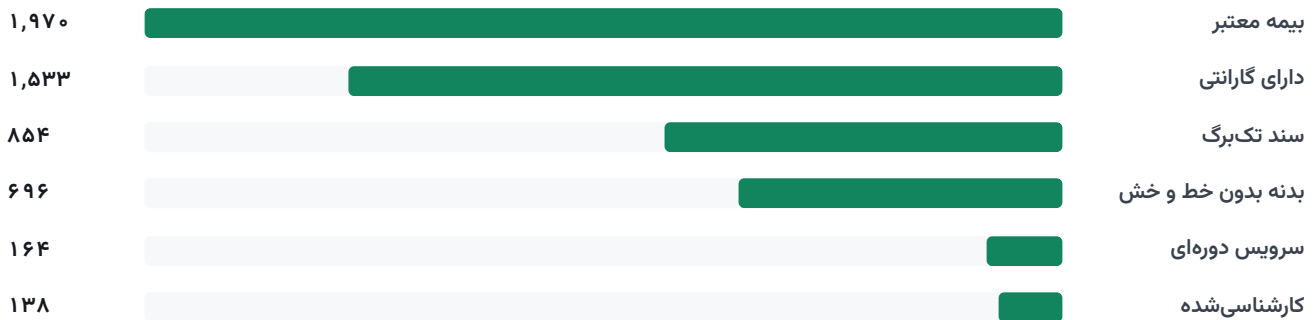
فروشنده‌ها چیزهایی را که اهمیت دارند در نثر دفن می‌کنند: «درب شاگرد آبرنگ»، «موتور تعویض شده»، «سند در رهن بانک». این‌ها ارزش و ایمنی خرید را تغییر می‌دهند و در هیچ فیلد ساختاریافته‌ای نیستند.

`enrich.py` یک **کار دسته‌ای** است که یک‌بار بعد از `ingest` اجرا می‌شود. یک اسکریپت کلیدواژه‌ای **همیشه** اول اجرا می‌شود؛ مدل زبانی ظرافت را روی آن اضافه می‌کند، وابستگی نیست. ادغام دو پاس، اتحاد است با نگه‌داشتن شدیدتر شدت در تعارض.



نکات مثبت استخراج شده

فقط نقاط فروش واقعی؛ شعار تبلیغاتی نمایشگاه و شماره تلفن نادیده گرفته می‌شود



کاربرد دو: تفسیر پرسش خریدار

query.py پرسش فارسی آزاد را به یک Intent ساختاریافته تبدیل می‌کند. دو مسیر با یک شکل خروجی:

- **مسیر قاعده‌محور** – همیشه اجرا می‌شود و هرگز خطا پرتاب نمی‌کند. بودجه، نوع بدنه، سوخت، گیربکس، برند و عبارات رایج کاربرد را پوشش می‌دهد. مبالغ فارسی («۸۰۰ میلیون»، «۲/۵ میلیارد») و ابهام سقف/کف («تا»، «زیر»، «حداکثر» در برابر «از»، «بالای») را تشخیص می‌دهد.
- **مسیر مدل زبانی** – دنباله بلند را می‌گیرد: عبارت‌بندی غیرمعمول و قیدهای ضمنی. خروجی **روی** نتیجه قاعده‌محور سوار می‌شود، پس کلید غایب یا پروکسی مرده پرسش را تضعیف می‌کند نه اینکه خالی کند.

پیش‌فرض‌های ضمنی کاربرد

یک کاربرد اعلام‌شده، اولویت‌هایی را ایجاد می‌کند که خریدار به زبان نیاورده: «خانواده» ← فضا، ایمنی، راحتی؛ «اولین ماشین» ← اقتصاد، مصرف کم، قطعات ارزان؛ «مسافرکشی» ← اقتصاد، قطعات ارزان، اطمینان‌پذیری، مصرف کم. این نداشت در هر دو مسیر یکسان اعمال می‌شود.

کاربرد سه: شباهت معنایی در بازیابی

برای نیت‌های مبهم («ماشین برای دختر دانشجو») هیچ کلمه‌ای از پرسش در هیچ آگهی‌ای وجود ندارد. مدل جاسازی جمله paraphrase-multilingual-MiniLM-L12-v2 پشت پرچم --embeddings در دسترس است، اما **پیش‌فرض ارسالی، مسیر جایگزین LSA است** – تجزیه مقادیر منفرد کوتاه‌شده روی همان ماتریس TF-IDF، که به هیچ داندودی نیاز ندارد.

وضعیت فعلی این نسخه، بی‌پرده

جدول غنی‌سازی کاملاً **قاعده‌محور است**. پوشه data/llm_cache/ وجود ندارد و هر ۲۱,۶۳۷ سطر غنی‌سازی دقیقاً از اسکنر کلیدواژه‌ای بازتولید می‌شود – یعنی هیچ‌چیز در پیکره ارسالی توسط یک مدل خوانده نشده است. محصول کار می‌کند: پرچم‌های ریسک، نکات مثبت و تفکیک امتیاز سلامت همه پر هستند. **این دقیقاً همان مسیر جایگزین است که وظیفه‌اش را انجام می‌دهد.**

وارد کردن لایه مدل زبانی، یک پاس حدود ۱۱/۹ هزار فراخوان ترتیبی هزینه دارد که بعد از آن کش رایگانش می‌کند. دو چیز این را یک گام عمدی می‌کند نه خودکار: enrich.py هیچ دسته‌بندی یا همزمانی ندارد پس پاس با ساعت اندازه‌گیری می‌شود؛ و --only-new تصمیم می‌گیرد چه چیزی را رد کند بر مبنای ستون model که ثبت می‌کند واقعاً چه چیزی آگهی را خوانده، نه اینکه چه چیزی درخواست شده بود.

درسی که در برچسب‌گذاری آموخته شد

برچسب‌زدن بر اساس **پرچم** به‌جای نتیجه، سطری را در هر اجرایی که مدل صرفاً **می‌توانست** فراخوانده شود `llm+rules` علامت می‌زد — از جمله اجراهایی که هر فراخوان چیزی برنگرداند. سطری که درباره خوانده‌شدنش دروغ بگوید، سطری است که `--only-new` برای همیشه ردش می‌کند.

پیکربندی نقطه پایانی مدل

هر نقطه پایانی سازگار با OpenAI را می‌پذیرد، پس همان کد روی یک پروکسی ایرانی (`AvalAI`، `Metis`، `Liara`، `GapGPT`)، `LLM_BASE_URL` یک `Ollama` محلی، یا مستقیم روی OpenAI کار می‌کند — فقط یک متغیر محیطی عوض می‌شود.

```
# backend/.env
LLM_BASE_URL=https://api.avalai.ir/v1 # یا http://localhost:11434/v1 برای Ollama
LLM_API_KEY=
LLM_MODEL=gpt-4o-mini
LLM_TIMEOUT=30
LLM_LIVE=true # false = هرگز شبکه فقط کش،
```

۱۲ جست‌وجو: اول بازیابی، بعد رتبه‌بندی

مسئله‌ای که این معماری برای حل آن ساخته شد

نسخه اول هیچ مرحله بازیابی نداشت. تابع `rank()` کل پیکره را روی ارزش/سلامت/تناسب امتیاز می‌داد، پس هر پرسشی که نمی‌فهمید، ارزان‌ترین خودرو نسبت به بازار را برمی‌گرداند – همان مشت خودرو، هر بار. جست‌وجوی «سراتو» یک BMW X4 برمی‌گرداند در حالی که ۲۷۳ کیا سراتو در پیکره دست‌نیافتنی نشسته بودند.

قیف بازیابی

پرسش فارسی خام

تاشدگی املایی: ی/ی، ک/ك، ZWNJ، ارقام فارسی و عربی، حذف اعراب

۱. تطبیق موجودیت –

فیلتر سخت

واژگان ۱۹۳ برند و ۷۵۱ مدل، مشتق‌شده از خود پیکره. «سراتو» یعنی کیا سراتو؛ هیچ چیز دیگری اجازه ظهور ندارد

▼ اگر موجودیتی تطبیق نداد

دروازه ۱: آیا این اصلاً پرسشی درباره خودرو است؟

فقط کلماتی که پیکره واقعاً استفاده می‌کند – نام صفت، یا واژگان رایج بین آگهی‌ها – به‌علاوه نیت شناخته‌شده خریدار

۳. معنایی

LSA با ۱۶۰ مؤلفه روی همان ماتریس TF-IDF، یا جاسازی جمله در صورت فعال بودن

۲. TF-IDF

دو نما: n-gram کاراکتری (۳ تا ۵) برای صرف فارسی، و توکن کلمه‌ای (۱ تا ۲) برای جداکردن کلمات – با وزن ۰/۶۵ به نفع کلمه

دروازه ۲: آیا این خودرو به بهترین گزینه به‌اندازه کافی نزدیک است؟

کف نسبی: متنی ۰/۴۰ و معنایی ۰/۵۵ از بالاترین امتیاز

حداکثر ۴۰۰ کاندید – رتبه‌بندی

ارزش * سلامت * تناسب فقط درون مجموعه بازیابی‌شده مرتب می‌کند – هرگز دیگر تصمیم‌گیرنده ربط نیست

چرا دو دروازه لازم بود

شباهت به‌تنهایی همیشه چیزی را اول قرار می‌دهد. بدون دروازه اول، «یخچال فریزر» و «فراری» پرسش خودرو حساب می‌شدند، چون یک نمایه جست‌وجو همیشه بهترین تطبیقش را برای هر ورودی پیدا می‌کند. بدون دروازه دوم، بازیابی ۴۰۰ سطر برتر را بدون توجه به امتیاز پاس می‌داد و نرمال‌سازی بهترین یک دسته بد را به ۱/۰ مطمئن بازمقیاس می‌کرد – این دقیقاً همان چیزی است که باعث شد جست‌وجوی «فراری» با یک مازراتی، یک ۲۰۷ و یک زوتی آریو پاسخ داده شود.

«این کلمه جایی وجود دارد» تقریباً هیچ شاهدهی نیست. توصیف‌های متن آزاد از یخچال، موتورسیکلت، خانه و فراری (در پیشنهاد معاوضه) حرف می‌زنند. پس دو واژگان جدا نگه داشته می‌شود: word_vocab فقط از فیلدهای هویتی (هر مدخل صفتی است که خودرویی واقعاً دارد) و word_df فراوانی سند روی متن کامل – که یک کلمه توصیف را فقط وقتی شاهد می‌شمارد که رایج باشد. اندازه‌گیری: «سقف» ۴/۹٪، «اقساط» ۳/۲٪، «شیشه» ۶/۷٪ آستانه یک درصد را رد می‌کنند؛ «یخچال» ۰/۶٪، «گوشی» ۰/۵٪ و «دوچرخه» ۰/۱٪ نه.

نتایج ارزیابی – اندازه‌گیری شده روی پیکره فعلی

tests/eval_search.py سی‌وهشت پرسش فارسی برچسب‌خورده را با قیدهایی که یک نتیجه درست باید برآورده کند جفت می‌کند و precision@5 گزارش می‌دهد. هر دو حالت زیر امروز روی همان پیکره ۲۲,۸۲۰ آگهی اجرا شدند.

دقت@۵ روی ۳۸ پرسش سخت (برچسب‌خورده با قید عینی)

مقایسه سیب با سیب: هر دو حالت روی همان پیکره امروز اجرا شده‌اند

بدون بازیابی	۴۲/۱٪ · ۱۴/۳۸
با بازیابی	۹۳/۲٪ · ۳۵/۳۸

عدد دوم: تعداد پرسش‌هایی که حداقل ۸۰٪ نتایجشان درست بوده است

سه شکست باقی‌مانده – و اینکه دو تای آن‌ها اشتباه تست‌اند

پرسش	انتظار	تحلیل
«یخچال فریزر»	هیچ نتیجه	اشتباه تست «وانت پراید یخچال‌دار» را بازیابی می‌کند. آن یک خودرو است، درست تطبیق داده شده، و انتظار تست از زمانی مانده که پیکره اتفاقاً چنین خودرویی نداشت
«خانه ویلایی»	هیچ نتیجه	هزینه واقعی «دوگانه کارخانه» را بازیابی می‌کند. n-gram کاراکتری همان چیزی است که به نمایه اجازه می‌دهد صرف فارسی را بدون ریشه‌یاب مدیریت کند، و بهایش این است که «خانه» داخل «کارخانه» تطبیق می‌خورد. عمداً اصلاح موردی نشده – یک فهرست توقف که تا سبز شدن ارزیابی تنظیم شود، برازش روی تست است نه روی زبان
«بین ۱ تا ۲ میلیارد»	بازه قیمت	سه از پنج نتیجه در بازه‌اند؛ باقی آگهی‌های بدون قیمتی هستند که برآوردشان لبه بازه را رد می‌کند – که رفتار عامدانه effective_price است

معامله‌ای که این معماری انجام می‌دهد، و پنهانش نمی‌کنیم

روی هشت پرسش عمداً مبهم («ماشین برای دختر دانشجو»، «ماشین لوکس و گران») نتیجه برعکس است: **بدون بازیابی ۷۲/۵٪**، با **بازیابی ۵۲/۵٪**. منطقی هم هست – رتبه‌بندی کل پیکره دقیقاً کاری است که برای پرسش مبهم درست است، و دروازه‌های صحت که «فراری» را متوقف می‌کنند، «دختر دانشجو» را هم محدود می‌کنند. مبادله عمدی است: دقت روی پرسش‌های عینی – که اکثریت قاطع ترافیک واقعی‌اند – در برابر ظرافت روی پرسش‌های سبک زندگی. مسیر بهبود، تقویت سیگنال معنایی است نه برداشتن دروازه‌ها.

مسیر جایگزین LSA و اینکه چرا در نخ پس‌زمینه ساخته می‌شود

برازش LSA روی پیکره زنده نزدیک ده ثانیه طول می‌کشد. قبلاً **تنبل** بود، یعنی داخل اولین پرسشی که به آن نیاز داشت برازش می‌شد – پس اولین جست‌وجوی مبهم هر اجرای سرور حدود چهارده ثانیه منتظر یک تجزیه ماتریسی می‌ماند. حالا:

- در زمان راه‌اندازی و در یک **نخ پس‌زمینه** ساخته می‌شود؛ API بلافاصله پاسخ می‌دهد.
- روی دیسک کش می‌شود (`data/lsa.npz`) زیر یک اثر انگشت از همان ماتریس TF-IDF که روی آن برازش شده.
- پیکره را دوباره ingest کنید و اثر انگشت دیگر تطبیق نمی‌خورد و کش بازبرازش می‌شود؛ دست‌نزد و راه‌اندازی فقط هزینه خواندن یک فایل حدود ۱۰۰ میلی‌ثانیه‌ای را می‌دهد.
- تا پایان کار نخ، پرسش‌ها فقط با ربط واژگانی پاسخ داده می‌شوند – کاری که بازیابی از قبل بلد است. **بازیابی هرگز خودش چیزی را برازش نمی‌کند.**
- وقتی نخ تمام شد، کش جست‌وجو پاک می‌شود؛ نتایجی که با ربط واژگانی تنها پاسخ داده شدند صادقانه‌اند، اما آن‌هایی نیستند که این نمایه حالا می‌دهد.

۱۳ فیلترها: در دوم ورود به جست‌وجو

نثر راه خوبی برای ورود است وقتی می‌دانید چه بگویید. راه بدی است وقتی نمی‌دانید، و اصلاً راهی نیست برای گفتن «حالا فقط اتوماتیک‌ها را نشان بده». پس همان جست‌وجو نوع دومی از ورودی می‌گیرد: ویژگی‌هایی که تیک می‌زنید.

۱۹ ویژگی قابل فیلتر، در پنج گروه

گروه	ویژگی‌ها	نوع
خودرو	برند · مدل · نوع بدنه	شمارشی (مدل به برند دامنه‌بندی شده)
قیمت	قیمت · فقط زیر قیمت بازار	بازه‌ای · بولی
وضعیت	سال تولید · کارکرد · وضعیت رنگ و بدنه · حداقل امتیاز سلامت · کارشناسی‌شده	بازه‌ای · نواری · بولی
فنی	گیربکس · سوخت · حجم موتور · مصرف سوخت	شمارشی · بازه‌ای
آگهی	رنگ بدنه · شهر · نوع فروشنده · منبع آگهی · فقط عکس‌دار	شمارشی · بولی

چهار چیزی که درباره این پنل ارزش توضیح دارد

یک · کاتالوگ مشتق می‌شود، هرگز دست‌نویس نیست

`/api/features` در زمان راه‌اندازی روی پیکره راه می‌رود و گزارش می‌دهد واقعاً چه چیزی آنجاست: **۹۶ برند، ۴۷۷ جفت برند+مدل، ۷۰ رنگ، ۵۶۳ شهر** — با برچسب‌گذاری از خود سطرها (`brand_fa` ، `model_fa` ، `body_type_fa`). برندی که در کراول فردا ظاهر شود بدون اینکه کسی فابلی را ویرایش کند قابل فیلتر است. نام انگلیسی برندها از اسلاگ لاتینی می‌آید که باما کنار فارسی منتشر می‌کند، پس ۸۷ تای آن‌ها بدون نگهداری دستی درست می‌مانند.

دو · نثر و تیک در یک نقطه به هم می‌رسند

هر دو یک `Intent` تولید می‌کنند و `passes_features` تنها دروازه‌ای است که هر دو از آن عبور می‌کنند — پس یک فیلتر نمی‌تواند تایپ‌شده یک معنا بدهد و کلیک‌شده معنایی دیگر. جایی که تداخل دارند، **تیک برنده است**: اگر پرسش گفته «اتوماتیک» و خریدار بعد تیک اتوماتیک را بردارد، خریدار برنده است — چون او می‌تواند یک چک‌باکس را ببیند و نمی‌تواند یک تفکیک‌کننده را ببیند.

سه · شمارش‌ها «یکی‌راکنارگذار» هستند

انتخاب خود هر ویژگی، پیش از شمارش مقادیر آن حذف می‌شود. اگر به روش بدیهی بشمارید، انتخاب «هیوندای» هر برند دیگری را صفر گزارش می‌کند، پس هرگز نمی‌توان برند دوم اضافه کرد و انتخاب چندگانه غیرقابل استفاده می‌شود. انجام درستش به پرسشی هم پاسخ می‌دهد که کسی نپرسیده بود: فیلتر کنید روی پراید بالای ۲۰ میلیارد تومان و پنل نشان می‌دهد آن بالا **چه چیزی** واقعاً وجود دارد — ۱۳۱ بی‌امو، ۷ تویوتا، ۵ بنز.

چهار • فیلتر قیمت روی `effective_price` اجرا می‌شود، نه قیمت آگهی

این کل استدلال پروژه است، اعمال شده روی یک اسلایدر: خودرویی که فروشنده‌اش «توافقی» نوشته، بر مبنای **برآورد ما** تطبیق داده و مرتب می‌شود. فیلتر روی بازه ۱/۵ تا ۲/۵ میلیارد، و ۴۴۱ مورد از ۵,۲۷۸ تطبیق خودروهایی هستند که هیچ قیمتی منتشر نمی‌کنند. هر انتخاب دیگری آن‌ها را دوباره غیرقابل فیلتر می‌کرد — که دقیقاً همان چیزی است که این محصول برای رفعش وجود دارد.

ویژگی‌هایی که عمداً بیرون گذاشته شدند

چرا این چهار فیلد فیلتر نمی‌شوند

فیلتری روی فیلدی به این نازکی، لحظه‌ای که لمس شود بیشتر پیکره را حذف می‌کند — و خریدار آن را «شما ماشین ندارید» می‌خواند، نه «ما داده نداریم»

۳۷,۳%		وضعیت شاسی
۶,۹%		امتیاز نمایشگاه
۶,۹%		مهلت بیمه
۱,۳%		قدرت موتور

هر سه فیلد اول در **امتیاز سلامت** حضور دارند، جایی که مقدار غایب فقط یعنی یک فاکتور کمتر

الگوی اعلانی Feature

هر ویژگی یک شیء است که برچسب‌های دوزبانه، تابع خواندن مقدار از یک سطر (`value_of`) و تابع آزمون آن سطر در برابر انتخاب کاربر (`matches`) را با هم حمل می‌کند. نگه‌داشتن هر دو روی یک شیء همان چیزی است که به `count_features` اجازه می‌دهد شمارش یکی‌راکنارگذار را به‌صورت عام انجام دهد، بدون یک شاخه `if` برای هر ویژگی.

شمارش‌شده از `features.FEATURES` : نوزده ورودی. `README` پروژه عدد هفده را ذکر می‌کند، که مربوط به پیش از افزوده‌شدن دو ویژگی آخر است.

۱۴ ارزش‌گذاری خودروی کاربر

هر مسیر دیگری در این برنامه از یک آگهی کراول‌شده شروع می‌شود. این یکی از یک آدم شروع می‌شود: خودرویش را به فارسی توصیف می‌کند، یا فرم پر می‌کند، یا هر دو – و همان ماشینی که ۲۲,۸۲۰ آگهی را قیمت‌گذاری می‌کند، خودروی او را قیمت می‌زند.

هیچ منطق جدیدی اینجا نیست – کار این ماژول «مونتاز و امتناع» است

برند و مدل از واژگان `lexicon.py`؛ سال، کارکرد، گیربکس و سوخت از همان تحلیل‌گر قاعده‌محور `query.py`؛ وضعیت رنگ از `normalize.infer_body_status` که برای خواندن دقیقاً همین نوع نثر نوشته شده بود؛ عبارات ریسک از `enrich.scan_text`؛ برآورد از همان رگرسور ارسالی و اطمینان از `pricing.confidence_for`.

امتناع به اندازه مونتاز اهمیت دارد

خودرویی که نتوانیم شناسایی کنیم، یا خودرویی بدون سال مدل، یک `status` صادقانه می‌گیرد و **هیچ عددی** – همان پاسخی که جست‌وجو وقتی می‌دهد که پرسش خودرویی را نام ببرد که موجودش نیستیم. یک ارزش‌گذاری ساختگی بدتر از نبود ارزش‌گذاری است: آدم آن‌طرف قرار است یک معامله واقعی را روی آن قیمت‌گذاری کند.

وضعیت	عدد می‌دهد؟	معنا
ok	بله	خودرو شناسایی شد، سال مشخص است، هم‌گروه کافی وجود دارد
unknown_car	خیر	نتوانستیم برند/مدل را به چیزی که در پیکره داریم حل کنیم
need_year	خیر	سال مدل غایب است؛ سن ورودی اصلی مدل قیمت است

دو تفاوت با یک آگهی کراول‌شده – که هر دو به‌صورت اطمینان‌پایین‌تر ظاهر می‌شوند

- بدون تیپ. یک فرم `trim_en` نمی‌پرسد، پس هم‌گروه مقایسه‌ای یک سطح درشت‌تر از یک آگهی شروع می‌شود.
 - شواهد نازک. هیچ‌کس قدرت موتور یا مصرف سوخت خودروی خودش را تایپ نمی‌کند، و ضریب شواهد اطمینان را تنزیل می‌دهد.
- نتیجه: یک ارزش‌گذاری خوب اینجا حوالی ۰/۵ تا ۰/۶ اطمینان می‌ایستد، نه ۰/۹ – و بازه قیمت متناسباً پهن‌تر است. این عدد صادقانه است.

جانهی مقادیر خالی: یک دسته‌ای خالی، خنثی نیست

`IMPUTED_FEATURES` شامل گیربکس، سوخت، نوع بدنه و شهر است. اگر کاربر آن‌ها را خالی بگذارد، رایج‌ترین مقدار پیکره جایگزین می‌شود – چون رشته `"unknown"` برای مدل یک دسته **معنادار** است که با آگهی‌های ناقص هم‌بسته است، نه یک مقدار خنثی. سال جلالی تایپ‌شده در فرم هم دقیقاً همان‌طور تبدیل می‌شود که در کراول: «۱۳۹۵» و «۲۰۱۶» یک خودرو یعنی.

هفت نقطه پایانی

نقطه پایانی	هدف
GET /api/search?q=...	جست‌وجوی زبان طبیعی، رتبه‌بندی‌شده و توضیح‌داده‌شده، صفحه‌بندی‌شده
GET /api/search?brands=...&price_max=...&sort=...	همان نقطه پایانی، فیلترشده بر اساس ویژگی — GET است تا نتیجه فیلترشده به صورت URL قابل اشتراک بماند
GET /api/features	کاتالوگ ویژگی قابل فیلتر، مشتق‌شده از پیکره
GET /api/car/{code}	یک آگهی با مقایسه‌ای‌ها و تفکیک امتیاز سلامت
GET /api/appraise?q=...	قیمت‌گذاری خودرویی که کاربر مالک آن است
GET /api/compare?codes=a,b	مقایسه کنار هم، حداکثر چهار خودرو
GET /api/stats	خلاصه پیکره — صفحه فرود را تغذیه می‌کند
GET /api/health	زنده‌بودن، وضعیت LLM، وضعیت کش و آمادگی نمایه معنایی

سه تغییری که جست‌وجو را سریع کردند

شبکه نتایج قبلاً صفحه‌بندی عددی داشت. هر دریافت باید می‌دانست چند صفحه وجود دارد، که یعنی مرتب‌کردن کل مجموعه نتیجه و ساختن بار توضیح‌داده‌شده برای هر خودروی آن، پیش از برگرداندن بیست‌وچهار تا — حدود نیم ثانیه برای مرور بدون فیلتر روی ۲۲,۸۲۰ آگهی، و پرداخت دوباره برای هر ورق‌زدن.

تغییر	مکانیزم	اثر
جداکردن مرتب‌سازی از توضیح	<code>order_all</code> کاندیدها را فقط با اعداد مرتب می‌کند (امتیاز سلامت از کش سطر می‌آید نه از پیمایش دوباره دوازده فاکتور)؛ <code>Ranking.page</code> فقط برای ۲۴ خودرویی که خواننده می‌بیند بار توضیح می‌سازد	۱۶۴ms ← ۵۰۷ms
کش‌کردن ترتیب	کلید کش: پرسش + فیلترها، اما نه پنجره. یک شبکه پیمایشی همان جست‌وجو را یک‌بار به‌ازای هر صفحه‌نمایش می‌پرسد؛ <code>limit</code> و <code>offset</code> فقط انتخاب می‌کنند کجای یک پاسخ از قبل محاسبه‌شده را ببینیم	۶ میلی‌ثانیه در پیمایش
ارسال یک‌باره شمارش ویژگی‌ها	شمارش‌ها کل مجموعه نتیجه را توصیف می‌کنند نه دسته را؛ ارسال ده‌ها کیلوبایت از آن‌ها در هر پیمایش فقط پل را عیناً بازترسیم می‌کرد	فقط <code>offset=0</code>

زمان پاسخ اندازه‌گیری‌شده در برابر پیکره زنده

اولین دریافت در برابر هر پیمایش بعدی — مقیاس لگاریتمی نیست، تفاوت واقعاً به این اندازه است

~۶۰۰ ms	مرور — اول
۶ ms	مرور — پیمایش
۶۰ ms	«سراتو سفید» — اول
۴ ms	«سراتو سفید» — پیمایش

آنچه از دست رفت، و چرا اهمیتی نداشت

شماره صفحه قابل پیوند. آن، اصلی‌ترین استدلال به نفع صفحه‌بندی عددی بود — چون URL منبع حقیقت این برنامه است. اما شماره صفحه هرگز چیزی نبود که ارزش پیوند داشته باشد: `?q=cerato&brands=kia&sort=price_asc` هنوز دقیقاً مثل قبل از نوار آدرس رفت‌وبرگشت می‌کند؛ اینکه کسی چقدر پیمایش کرده نه، و نباید هم بکند. یک دکمه واقعی زیر حسگر پیمایش می‌نشیند برای هرکس که با صفحه‌کلید یا صفحه‌خوان حرکت می‌کند، که یک حسگر پیمایش نمی‌تواند به او خدمت کند.

لایه رابط کاربری

۱۹ React با TypeScript و Tailwind، راست‌به‌چپ فارسی به‌عنوان پیش‌فرض و کلید تغییر به انگلیسی. جهت روی خود سند تنظیم می‌شود تا نوار پیمایش و کنترل‌های بومی فرم هم برگردند، نه فقط چیدمان خودمان.

مؤلفه	نقش
Hero · SearchBar	صفحه فرود با آمار زنده پیکره و پرسش‌های نمونه
FilterPanel · ActiveFilters	پنل ۱۹ ویژگی با شمارش زنده و چیپ‌های فیلتر فعال قابل حذف
CarCard · Price · Health	کارت نتیجه: قیمت مؤثر، نشان اختلاف بازار، نوار سلامت و دلایل رتبه
CarDetail	کشوی جزئیات با تفکیک کامل امتیاز سلامت و آگهی‌های مقایسه‌ای
AppraiseView	نمای «قیمت ماشین من» — نمای دوم روی همان پیکره، نه اپلیکیشن دوم
CarArt · Logo	گرافیک برداری درون‌خطی که تم را به ارث می‌برد؛ بدون درخواست دارایی دوم
ThemeToggle · i18n	تم روشن/تیره و دو زبان کامل با ۳۵۷ خط رشته

URL منبع حقیقت است: هر جست‌وجوی فیلترشده قابل بارگذاری مجدد، قابل اشتراک و قابل بازگشت با دکمه‌های خود مرورگر است. تنها وابستگی زمان اجرای فرانت‌اند، React و React DOM است — بدون کتابخانه UI، بدون کتابخانه نمودار، بدون مدیریت وضعیت.

۱۶ تصمیمات معماری و دلیل هرکدام

هر ردیف این جدول یک تصمیمی است که می‌شد جور دیگری گرفته شود. ستون آخر دلیل است، نه توجیه – و جایی که تصمیم از یک اندازه‌گیری آمده، عدد آورده شده.

تصمیم	جایگزین رد شده	دلیل
API عمومی JSON، نه خراش HTML	مرورگر بی‌سر + سلکتور CSS	بدون مرورگر بی‌سر، بدون سلکتور شکننده. هر سایت یک آداپتور <code>Source</code> است که فقط دو چیز را مالک است، پس افزودن سایت هرگز به منطق محصول دست نمی‌زند
ذخیره رکورد خام دست‌نخورده	نرمال‌سازی حین کراول	بازپردازش پس از اصلاح یک باگ نرمال‌سازی هرگز به معنای کراول دوباره نیست – برای دیوار تفاوت بین یک دقیقه و چند ساعت
کراولر سرعت سایت را خودش پیدا می‌کند	تعداد کارگر ثابت + سه تلاش مجدد	اندازه‌گیری: پنج کارگر ۱/۲ برابر سریع‌تر بود و ۸ آگهی از ۴۸ گم کرد. یک ۴۲۹ کل منبع را کند می‌کند و موفقیت آن را به‌آرامی واپاشی می‌دهد؛ نرخ ازدست‌رفتن از ۴/۲٪ به ۱/۰٪ رسید
قیمت توافقی <code>NULL</code> است، هرگز صفر	ذخیره صفر	یک صفر هدف رگرسیون را مسموم می‌کند. و این سطرها دقیقاً همان‌هایی‌اند که محصول برای خدمت به آن‌ها وجود دارد
آموزش روی <code>log(price)</code> با زیان خطای مطلق	رگرسیون روی قیمت خام با MSE	قیمت‌ها از حدود ۱۰۰ میلیون تا ۴۵ میلیارد تومان کشیده‌اند؛ بدون تبدیل لگاریتمی خطای یک بنز خطای هر پراید را غرق می‌کرد. زیان مطلق در برابر قیمت‌های پرت باقی‌مانده مقاوم است
اطمینان از عمق مقایسه‌ای واقعی	یک درصد اطمینان ثابت	یک هم‌گروه نازک باید اطمینان را به‌شکل مرئی پایین بیاورد، نه اینکه بی‌صدا یک عدد را بلوف کند. عددی که کسی نتواند بررسی کند، بدتر از نبود عدد است
هویت بین‌منبعی آموخته می‌شود	جدول نگاشت دست‌نویس	باما <code>brand='peugeot'</code> را کنار <code>brand_fa='پژو'</code> منتشر می‌کند، پس نگاشت را می‌آموزاند. جدول دستی با هر برند جدید کهنه می‌شد
تشخیص تکراری محافظه‌کارانه	ادغام سخاوتمندانه بر اساس شباهت	یک ادغام اشتباه یک آگهی واقعی را پنهان می‌کند، که بدتر از نشان‌دادن دوباره یک خودرو است
خودروی صفرکیلومتر خارج از دامنه	گنجاندن همه چیز	بازار خودروی نوی ایران دوقیمتی است: قیمت کارخانه/حواله و قیمت بازار آزاد هم‌زمان برای یک مدل وجود دارند، پس «ارزش بازار» یک عدد نیست و هر مقایسه‌ای علیه آن، معامله شبح تولید می‌کند. موتور سلامت هم در صفر کیلومتر بی‌معناست. حدود ۴,۰۰۰ آگهی حذف شد؛ با <code>--include-new</code> قابل بازگرداندن
هر خروجی LLM روی دیسک کش و کامیت می‌شود	فراخوان زنده در مسیر درخواست	مسیر درخواست هرگز منتظر مدل نمی‌ماند و برنامه با شبکه قطع درست کار می‌کند. یک قابلیت هوش مصنوعی که صفحه را با خودش پایین بیاورد، بدتر از نبود آن قابلیت است
SQLite، نه PostgreSQL	پایگاه‌داده سرویس‌دهنده	چند هزار سطر در یک فایل کامیت‌شده یعنی بازبین مخزن را کلون می‌کند و اجرا می‌کند – بدون هیچ پایگاه‌داده‌ای که باید تدارک دیده شود
رتبه‌بندی در پایتون، نه در SQL	امتیازدهی داخل پرس‌وجو	کل پیکره چند هزار سطر است و در حافظه جا می‌شود. نگه‌داشتن منطق رتبه‌بندی در پایتون خوانا آن را قابل توضیح نگه می‌دارد؛ در SQL استدلال ناپدید می‌شد
یک دروازه فیلتر برای هر دو ورودی	مسیر جدا برای نثر و برای تیک	یک فیلتر نمی‌تواند تایپ‌شده یک معنا بدهد و کلیک‌شده معنایی دیگر. جایی که تداخل دارند تیک برنده است، چون کاربر چک‌باکس را می‌بیند و تفکیک‌کننده را نمی‌بیند
پیش‌فرض ارسالی، LSA است نه جاسازی	مدل جاسازی جمله به‌عنوان پیش‌فرض	مدل جاسازی روی پرسش‌های مبهم وزنش را نمی‌کشد و به دانلود نیاز دارد. LSA به هیچ دانلودی نیاز ندارد و تضمین آفلاین فاز اول را نگه می‌دارد
مسیر جایگزین قطعی برای هر مسیر AI	وابستگی سخت به مدل	پروکسی خاموش: امتیازدهی سلامت به قواعد برمی‌گردد و تفکیک پرسش به تحلیل‌گر قطعی فارسی. اثبات عملی: کل پیکره ارسالی در همین حالت ساخته شده است

الف · قیمت

- **برآورد قیمت بازار برای هر آگهی** – شامل ۲,۵۹۸ آگهی که فروشنده هیچ عددی نگذاشته است.
- **نشان اختلاف قیمت** – «۱۲٪ زیر قیمت بازار» با آستانه‌های مشخص: زیر ۸٪ = معامله خوب، بالای ۱۲٪ = گران.
- **بازه اطمینان** – پهنای بازه از خطای واقعی اندازه‌گیری شده مدل و اطمینان همان آگهی می‌آید، نه از یک درصد ثابت.
- **آگهی‌های مقایسه‌ای قابل بازرسی** – تا شش آگهی هم‌مدل و نزدیک‌ترین سال و کارکرد، با پیوند به منبع اصلی، در کنشوی جزئیات.
- **عمق و سطح هم‌گروه** – نمایش اینکه برآورد روی تیپ، مدل یا سراسری ایستاده و چند مقایسه‌ای واقعی پشتش هست.
- **هشدار حواله و پیش‌فروش** – ۸۲۴ آگهی که عدد تبلیغ‌شده‌شان ودیعه است، به‌جای معامله شبج، هشدار می‌گیرند.
- **محافظت از قیمت جانگهدار** – عدد منتشرشده زیر ۲۰ میلیون تومان یا زیر ۱۵٪ برآورد خود خودرو، بی‌اعتماد شمرده می‌شود.

ب · سلامت

- **امتیاز ۰ تا ۱۰۰ با ده فاکتور**، هرکدام با برچسب دوزبانه و تأثیر عددی صریح – هیچ جعبه سیاهی نیست.
- **استنتاج وضعیت بدنه از نثر وقتی منبع فیلد را اعلام نمی‌کند**، با تطبیق بدترین‌اول.
- **سیگنال‌های اختصاصی هر منبع** – وضعیت شاسی از شیپور، مهلت بیمه از دیوار، کارشناسی از باما و کارنامه؛ غیاب هرکدام جریمه نیست.
- **استخراج پرچم ریسک از متن آگهی** – هفت دسته با سه سطح شدت؛ ۱,۸۸۰ آگهی دست‌کم یک پرچم دارند.
- **کارکرد نامتناسب با سن** – هم بالا بودن و هم پایین بودن غیرعادی، هر دو سیگنال‌اند.

ج · جست‌وجو و کشف

- **جست‌وجوی زبان طبیعی فارسی** با تحمل غلط املائی، انواع ی/ک، ZWNJ و ارقام فارسی و عربی.
- **تطبیق موجودیت به‌عنوان فیلتر سخت** – «سراتو» فقط کیا سراتو برمی‌گرداند.
- **پاسخ خالی صادقانه** – پرسشی که به هیچ خودرویی مربوط نیست، «چنین خودرویی نداریم» می‌گیرد نه ارزان‌ترین موجودی؛ با تفکیک نوع نتیجه‌نداشتن (`unknown_car` / `constraints`).
- **۱۹ فیلتر مشتق‌شده از پیکره** با شمارش زنده یکی‌راکنارگذار.
- **هفت ترتیب مرتب‌سازی** – پیشنهاد کلی، ارزان‌ترین، گران‌ترین، جدیدترین، کم‌کارکردترین، سالم‌ترین، بیشترین تخفیف نسبت به بازار.
- **شبکه پیمایشی با حسگر پیش‌بارگذاری** و دکمه واقعی زیر آن برای دسترسی با صفحه‌کلید و صفحه‌خوان.
- **دلایل روی هر کارت** – تا پنج دلیل که چرا این خودرو این رتبه را گرفت.
- **مقایسه کنار هم** تا چهار خودرو.

د · ابزار فروشنده

- **«قیمت ماشین من»** – توصیف فارسی آزاد یا فرم، با همان موتور قیمت.
- **نمایش آنچه تفکیک‌کننده فهمید** – فرم از پیش پرشده تا کاربر بتواند اصلاح کند، نه اینکه حدس بزند چرا عدد عجیب است.
- **امتناع صادقانه** – بدون شناسایی خودرو یا بدون سال مدل، هیچ عددی داده نمی‌شود.
- **آگهی‌های زنده مشابه** – تا ۱۲ خودروی نزدیک با پنجره دو ساله سال مدل.

ه . محصول و تجربه

- دوزبانه کامل فارسی راست به چپ و انگلیسی، با تنظیم جهت روی خود سند.
- تم روشن و تیره با پالت جداگانه برای هر کدام.
- URL منبع حقیقت – هر جست و جوی فیلترشده قابل اشتراک و قابل بازگشت با دکمه های مرورگر.
- نشان منبع روی هر کارت و پیوند مستقیم به آگهی اصلی – کاپوت خودش را جایگزین منبع نمی کند.
- پیوند آگهی های چندسایتی – ۱,۰۵۸ آگهی که روی بیش از یک سایت اند، به هم متصل شده اند.

۱۸ ارزش افزوده نسبت به پروژه‌های مشابه

مقایسه زیر بر مبنای سطح محصول رو به خریدار در جریان دیدن یک آگهی انجام شده است – نه بر مبنای آنچه ممکن است پشت صحنه هر شرکتی وجود داشته باشد.

قابلیت	کاپوت	دیوار	باما	شیپور	کارنامه	تجمیع‌کننده عمومی
تجمیع چند سایت در یک نتیجه	بله	خیر	خیر	خیر	خیر	بله
برآورد قیمت بازار برای هر آگهی	بله	خیر	جزئی	خیر	جزئی	خیر
قیمت‌گذاری آگهی «توافقی»	بله	خیر	خیر	خیر	خیر	خیر
فیلتر و مرتب‌سازی آگهی بدون قیمت	بله	خیر	خیر	خیر	خیر	خیر
بازه اطمینان روی برآورد	بله	خیر	خیر	خیر	خیر	خیر
امتیاز سلامت تفکیک‌شده	بله	خیر	خیر	خیر	گزارش کارشناسی	خیر
استخراج ریسک از متن آگهی	بله	خیر	خیر	خیر	خیر	خیر
جست‌وجوی زبان طبیعی فارسی	بله	جزئی	خیر	خیر	خیر	جزئی
رتبه‌بندی بر مبنای نیت خریدار	بله	خیر	خیر	خیر	خیر	خیر
نمایش دلیل هر رتبه	بله	خیر	خیر	خیر	خیر	خیر
تشخیص آگهی تکراری بین سایت‌ها	بله	—	—	—	—	جزئی
پاسخ خالی صادقانه به پرسش نامربوط	بله	جزئی	جزئی	جزئی	—	خیر

«جزئی» یعنی قابلیت به شکلی محدود، در بخشی از موجودی، یا به‌عنوان یک ابزار جدا از جریان جست‌وجو وجود دارد. «—» یعنی برای آن محصول موضوعیت ندارد.

پنج تمایز ساختاری

۱. قیمت‌گذاری نقطه کور بازار

این تنها قابلیت است که در جدول بالا هیچ‌کس دیگری ندارد و هسته استدلال محصول است. ۱۱/۵٪ عرضه که برای هر خریدار بودجه‌محور نامرئی است، اینجا مرئی می‌شود – و نه فقط با یک عدد، بلکه با یک بازه اطمینان که می‌گوید چقدر باید به آن عدد اعتماد کرد.

۲. هر عدد قابل بازخواست است

امتیاز سلامت به ده فاکتور با تأثیر عددی شکسته می‌شود. برآورد قیمت به آگهی‌های واقعی لینک می‌شود. رتبه به فهرست دلایل. رقبا یا عددی نمی‌دهند یا عددی می‌دهند که پشتش قابل دیدن نیست.

۳. اطمینان تفکیک‌شده به‌جای عدد ترکیبی

یک برآورد باما با ۵۹۵٪ اطمینان و یک برآورد کارنامه با ۲۱۸٪ ارائه می‌شوند، چون شواهد پشتشان واقعاً به همین اندازه فرق دارد. انتشار یک عدد واحد برای هر دو، همان «عددی که کسی نمی‌تواند بررسی کند» بود.

۴. دو در ورودی که نمی‌توانند اختلاف داشته باشند

نثر و تیک هر دو یک Intent تولید می‌کنند و از یک دروازه رد می‌شوند. در محصولاتی که این دو مسیر جدا پیاده می‌شوند، «اتوماتیک» تایپ‌شده و «اتوماتیک» تیک‌خورده به‌مرور دو معنای متفاوت پیدا می‌کنند.

۵. واژگان خودآموز

۱۹۳ نام برند و ۷۵۱ نام مدل از خود پیکره مشتق می‌شوند. برند یا مدلی که در کراول فردا ظاهر شود، بدون ویرایش هیچ فایلی هم قابل جست‌وجو و هم قابل فیلتر است. جدول‌های دست‌نویس با هر مدل جدید بازار کهنه می‌شوند.

۶. صداقت به‌عنوان ویژگی محصول

«چنین خودرویی نداریم» یک پاسخ درست است. برگرداندن ارزان‌ترین موجودی برای پرسشی که نفهمیده‌ایم، اعتماد را جای دقت می‌فروشد. کاپوت نوع نتیجه‌نداشتن را هم تفکیک می‌کند تا رابط کاربری بتواند دلیل واقعی را بگوید.

۱۹ مرجع کامل دستورات ترمینال

پیش‌نیازها: Python ۳/۱۲ به بالا و Node ۲۰ به بالا. همه دستورها با `.venv/bin/python` نوشته شده‌اند تا فارغ از اینکه محیط مجازی را فعال کرده باشید یا نه کار کنند.

۰۱ راه‌اندازی اولیه بک‌اند

```
cd backend
```

```
./setup.sh
```

می‌نویسد `.env`، محیط مجازی می‌سازد، وابستگی‌ها را نصب می‌کند #

چرا `setup.sh` وجود دارد

روی سیستم‌های دیبیا و اوبونتو که `python3` را بدون بسته `python3-venv` عرضه می‌کنند، دستور `python3 -m venv` صریحاً شکست می‌خورد — نه `ensurepip` هست و نه اغلب `pip` سیستمی. اسکریپت این حالت را تشخیص می‌دهد و `pip` را با نصب‌کننده رسمی PyPA داخل محیط مجازی راه‌اندازی می‌کند، پس نه `sudo` لازم است و نه بسته سیستمی. اگر ترجیح می‌دهید در سطح سیستم حلش کنید: `sudo apt install python3-venv python3-pip`

۰۲ کراول — هر منبع جداگانه

منابع هزینه زمانی بسیار متفاوتی به ازای هر خودرو دارند، پس ارزش دارد جداگانه اجرا شوند.

```
# باما — ۱۰۰ صفحه خوراک عمومی + کراول لایه‌بندی‌شده ۲۵ برند + ۵۰۰ رکورد جزئیات
.venv/bin/python -m app.crawl.run --sources bama --pages 100 --details 500
```

```
# شیپور — ارزان‌ترین منبع؛ حدود ۱۲ آگهی در ثانیه چون مشخصات داخل پاسخ جستوجوست
.venv/bin/python -m app.crawl.run --sources sheypoor --pages 600
```

```
# کارنامه — کوچک و صادق؛ ۱۸ صفحه بیشتر ندارد
.venv/bin/python -m app.crawl.run --sources karnameh --pages 25
```

```
# قابل ازسرگیری دیوار — کند (حدود ۰.۷ آگهی در ثانیه)، اما با
.venv/bin/python -m app.crawl.run --sources divar --pages 30 --append
```

پرچم	پیش‌فرض	کار
<code>--sources</code>	bama	فهرست جداشده با کاما؛ شناخته‌شده‌ها: bama, divar, karnameh, sheypoor
<code>--pages</code>	60	تعداد صفحه به ازای هر خوراک
<code>--details</code>	500	تعداد رکورد جزئیات باما (مشخصات فنی، سبک زندگی، سابقه نمایشگاه)
<code>--cities</code>	۱۲ شهر	شناسه شهرهای دیوار، جداشده با کاما
<code>--concurrency</code>	2	درخواست‌های جزئیات دیوار در پرواز؛ بالاتر بردن محدودیت انفجاری دیوار را فعال می‌کند و آگهی گم می‌شود
<code>--append</code>	خاموش	ازسرگیری: رکوردهای موجود را نگه می‌دارد و شناسه‌های گرفته‌شده را رد می‌کند

مثال‌های بیشتر

```
# چند منبع در یک اجرا (پشت سر هم، نه موازی)
.venv/bin/python -m app.crawl.run --sources bama,divar --pages 60

# فقط تهران و کرج از دیوار
.venv/bin/python -m app.crawl.run --sources divar --pages 25 --cities 1,2

# ادامه یک کراول نیمه‌تمام دیوار بدون پرداخت دوباره درخواست جزئیات
.venv/bin/python -m app.crawl.run --sources divar --pages 40 --append --concurrency 2
```

۳. بارگذاری در پایگاه‌داده

```
.venv/bin/python -m app.ingest          # نرمال‌سازی SQLite
.venv/bin/python -m app.ingest --sources bama,sheypoor # فقط این منابع
.venv/bin/python -m app.ingest --include-new           # خودروهای صفرکیلومتر را هم وارد کن
```

این مرحله بازسازی کامل از فایل‌های خام است: تفکیک، حل هویت برند و مدل، پیوند آگهی‌های تکراری، حذف صفرکیلومترها و در پایان **هرس سطرهای کهنه** – هر آگهی که در این پاس تولید نشده از جدول حذف می‌شود، وگرنه `INSERT OR REPLACE` بی‌صدا آن‌ها را انباشته می‌کرد و هر آمار پیکره منحرف می‌شد.

۴. آموزش و امتیازدهی مدل قیمت

```
.venv/bin/python -m app.pricing --train # آموزش + امتیازدهی؛ خطا را به تفکیک منبع چاپ می‌کند
.venv/bin/python -m app.pricing        # فقط امتیازدهی مجدد با مدل موجود روی دیسک
```

خروجی نمونه از اجرای امروز روی پیکره فعلی:

```
loaded 22820 listings
trained on 15424 rows | holdout MAPE=14.4% median APE=7.6% R2(log)=0.929
bama      n=1768  MAPE=10.2%  median APE=6.0%
sheypoor  n=1653  MAPE=17.4%  median APE=9.1%
divar     n=373   MAPE=18.5%  median APE=10.5%
```

```
karnameh n=63 MAPE=26.1% median APE=15.2%
scored 22820 listings (22732 with an estimate)
gave a price to 2598 listings that had none
```

۵. غنی‌سازی – استخراج سیگنال ریسک

```
.venv/bin/python -m app.enrich --only-new # آنچه از قبل انجام شده را رد می‌کند
.venv/bin/python -m app.enrich --no-llm # فقط اسکن کلیدواژه‌ای، بدون هیچ فراخوان مدل
.venv/bin/python -m app.enrich --limit 500 # سقف تعداد آگهی پردازش‌شده (۰ = همه)
```

درباره --only-new

«از قبل انجام‌شده» به این بستگی دارد که این اجرا چه چیزی تولید می‌کند. سطری که فقط با قواعد اسکن شده، برای یک اجرای LLM کار تمام‌شده نیست؛ رد کردنش آن را برای همیشه از دسترس پاس گران‌تر پنهان می‌کند. پس با `--no-llm` هر سطر موجود رد می‌شود، و بدون آن فقط سطرهایی که ستون `model` آن‌ها `llm+rules` است.

۶. اجرای سرور

```
.venv/bin/uvicorn app.main:app --reload # http://localhost:8000
```

```
cd frontend
npm install
npm run dev # http://localhost:5173
```

بک‌اند باید روی پورت ۸۰۰۰ در حال اجرا باشد – سرور توسعه مستقیم آن را صدا می‌زند و CORS به `localhost:5173` محدود شده است. اگر API را جای دیگری سرو می‌کنید، با `VITE_API_BASE` بازنویسی کنید.

۷. تست و ارزیابی

```
cd backend
.venv/bin/python -m pytest tests/ -q # تست ۲۱۸

.venv/bin/python -m tests.eval_search # دقت@۵ روی ۳۸ پرسش برچسب‌خورده
.venv/bin/python -m tests.eval_search --verbose # نتایج پرسش‌های شکست‌خورده را هم نشان بده
.venv/bin/python -m tests.eval_search --baseline # رفتار فاز اول: رتبه‌بندی کل پیکره بدون ارزیابی
.venv/bin/python -m tests.eval_search --embeddings # LSA با مدل جاسازی جمله به جای
.venv/bin/python -m tests.eval_search -k 10 # دقت@۱۰ به جای ۵@
```

۸. داکر – یک دستور برای کل پشته

```
# backend/data/ (cars.db و price_model.pkl) با یک پیکره ساخته‌شده در
docker compose up --build
```

رابط کاربری روی `http://localhost:3080`؛ nginx مسیر `/api` را به بک‌اند پروکسی می‌کند. تنظیمات مدل زبانی از `backend/.env` یا متغیرهای محیطی Compose می‌آید. برای در دسترس بودن API روی خود میزبان، خط `ports: ["8000:8000"]` را زیر `backend` در

۹. دنباله کامل، از صفر تا سرور

```
cd backend && ./setup.sh

.venv/bin/python -m app.crawl.run --sources bama --pages 100 --details 500
.venv/bin/python -m app.crawl.run --sources sheypoor --pages 600
.venv/bin/python -m app.crawl.run --sources karnameh --pages 25
.venv/bin/python -m app.crawl.run --sources divar --pages 30 --append

.venv/bin/python -m app.ingest
.venv/bin/python -m app.pricing --train
.venv/bin/python -m app.enrich --only-new

.venv/bin/python -m pytest tests/ -q
.venv/bin/python -m tests.eval_search

.venv/bin/uvicorn app.main:app --reload
```

۲۰ کیفیت مهندسی، تست و استقرار

۱۲	۳۸	۱/۲s	۲۲۴
وابستگی مستقیم پایتون	پرسش برچسب‌خورده در ارزیابی جست‌وجو	زمان اجرای کل مجموعه تست	تست، همه سبز

پوشش تست

فایل	تست	چه چیزی را تثبیت می‌کند
test_features.py	۴۵	کاتالوگ مشتق‌شده، دروازه فیلتر، شمارش یکی‌راکنارگذار، effective_price
test_appraise.py	۳۱	تفکیک توصیف، ادغام فرم و نثر، مسیرهای امتناع، جانهی مقادیر
test_normalize.py	۲۴	تقویم جلالی، ارقام فارسی و عربی، «صفر کیلومتر»، اسلاگ URL، درجه‌بندی رنگ
test_lexicon.py	۲۲	تاشدگی املائی، تطبیق موجودیت، تحمل غلط، ساخت واژگان از پیکره
test_search.py	۲۰	دو دروازه صحت پرسش، کف نسبی، حالت‌های بازیابی، کش LSA
test_divar.py	۱۷	پیمایش ویجت، تفکیک سال دوتقویمی، استنتاج وضعیت بدنه از متن
test_query.py	۱۷	تفکیک بودجه، سقف در برابر کف، پیش‌فرض‌های ضمنی کاربر
test_sheypoor.py	۱۱	تأیید دسته از بردکرامب، بازکردن vip، تفکیک قیمت توافقی، وضعیت شاسی
test_pagination.py	۱۱	صفحه به‌عنوان پنجره‌ای بر یک ترتیب سراسری، رفتار خارج از محدوده
test_rank_sort.py	۱۰	هفت ترتیب مرتب‌سازی، رفتار مقدار غایب («آخر، نه صفر»)
test_karnameh.py	۱۰	استخراج URL، buildId، صفحه یک، رکوردهای تایپ‌شده، عدم ثبت prepayment

ارزیابی، نه فقط تست واحد

«جست‌وجو دقیق نیست» یک عدد نیست. tests/eval_search.py آن را به عدد تبدیل می‌کند: هر مورد یک پرسش واقع‌گرایانه فارسی را با قیدهایی که یک نتیجه درست باید برآورده کند جفت می‌کند و precision@5 گزارش می‌شود. موارد روی موجودیت‌هایی پایه‌گذاری شده‌اند که وجودشان در پیکره تأیید شده (کیا سراتو: ۲۷۳ آگهی، پژو ۲۰۶: ۵۵، رنو تندر ۹۰: ۶۷۱)، پس شکست یعنی جست‌وجو آن‌ها را از دست داده، نه اینکه داده غایب است. پرچم --baseline رفتار فاز اول را نگه داشته تا هر تغییری در جست‌وجو در برابر همان خط‌مبنا سنجیده شود.

جزء	پیکربندی
بک‌اند	تصویر داکر با <code>backend/data</code> به‌عنوان حجم پایدار؛ FastAPI روی <code>uvicorn</code>
فرانت‌اند	بیلد Vite سروشده با <code>nginx</code> روی پورت ۳۰۸۰؛ <code>nginx</code> مسیر <code>/api</code> را به بک‌اند پروکسی می‌کند
پیکربندی	<code>env_file</code> اختیاری به‌علاوه متغیرهای محیطی <code>Compose</code> ؛ همه با پیش‌فرض معقول
CORS	فهرست مجاز از <code>CORS_ORIGINS</code> یا پیش‌فرض‌های <code>Vite</code> و <code>Compose</code> ؛ فقط متد <code>GET</code>
سلامت سرویس	<code>/api/health</code> تعداد آگهی بارگذاری‌شده، آمادگی نمایه، حالت سیگنال معنایی و وضعیت کنش مدل را برمی‌گرداند

الگوهای تکرارشونده در کد

- یک تعریف برای هر مفهوم. `effective_price` در `features.py` زندگی می‌کند و `rank.py` آن را دوباره صادر می‌کند، چون فیلتر و رتبه‌بند باید قیمت را یکسان بخوانند. `confidence_for` عمومی است چون `appraise.py` باید از همان مسیر به همان عدد برسد.
- مسیر خواندن واحد. `_FULL_SELECT` تنها پرس‌وجویی است که جست‌وجو، جزئیات و مقایسه از آن می‌خوانند، پس نمی‌توانند با هم اختلاف پیدا کنند.
- مسیرهای فایل یک‌بار نام‌گذاری می‌شوند. `config.py` هر مصنوع روی دیسک را مالک است؛ قبلاً چهار نسخه از همان `parents[1] / "data"` وجود داشت و دو املائی مستقل از محل فایل خام، پس کراولر یک مسیر می‌نوشت و `ingest` مسیری دیگر می‌خواند.
- مهاجرت ستون در `init_db`. `CREATE TABLE IF NOT EXISTS` بعد از وجود جدول بی‌اثر است، پس ستون تازه‌افزوده هرگز به یک `cars.db` قدیمی نمی‌رسید و اولین `upsert` شکست می‌خورد.
- یک منبع مرده کل کراول را متوقف نمی‌کند. حلقه `run()` هر منبع را جدا در `try` می‌گیرد و لاگ می‌کند.

۲۱ محدودیت‌ها و مسیر پیش رو

چیزی که یک گزارش فنی باید بگوید، نه فقط آنچه کار می‌کند. هر مورد زیر یک محدودیت شناخته شده است، نه یک نقص کشف نشده.

محدودیت‌های داده

محدودیت	اثر	مسیر رفع
پیکره یک عکس لحظه‌ای است	کراول یک‌بار در شهریور ۱۴۰۵ انجام شده؛ قیمت‌ها در بازاری با تورم بالا کهنه می‌شوند	کراول زمان‌بندی شده تدریجی با <code>append</code> - - ؛ زیرساخت ازسرگیری از قبل هست
سهم دیوار کم است	۱,۸۵۷ آگهی از بزرگ‌ترین سایت آگهی کشور – چون هر آگهی یک درخواست جزئیات لازم دارد	کراول تدریجی طولانی‌مدت؛ محدودیت زمان است نه فنی
مشخصات فنی فقط از باما	حجم موتور و مصرف سوخت روی ۳۹٪ پیکره؛ همین شکاف، اختلاف خطای ۱۰/۲٪ در برابر ۲۶٪ را توضیح می‌دهد	غنی‌سازی از یک کاتالوگ خودرو بر اساس برند+مدل+سال، جدا از آگهی
سوگیری انتخاب در آگهی «توافقی»	مدل روی آگهی‌های قیمت‌دار آموزش می‌بیند و روی بی‌قیمت‌ها تعمیم می‌دهد؛ اگر پنهان کردن قیمت با ویژگی خودرو همبسته باشد، برآورد سوگیری دارد	اعتبارسنجی روی آگهی‌هایی که بعداً قیمت‌دار شدند – نیازمند کراول سری زمانی
لایه LLM هنوز اجرا نشده	هر ۲۱,۶۳۷ سطر غنی‌سازی قاعده‌محورند؛ ظرافت متن استخراج نشده است	یک پاس حدود ۱۱/۹ هزار فراخوان؛ افزودن دسته‌بندی و همزمانی به <code>enrich.py</code>

محدودیت‌های الگوریتمی

- پرسش‌های مبهم پسرقت داشته‌اند. دقت روی هشت پرسش سبک‌زندگی از ۷۲/۵٪ (بدون بازیابی) به ۵۲/۵٪ رسیده. دروازه‌هایی که «فراری» را متوقف می‌کنند «دختر دانشجو» را هم محدود می‌کنند. رفع درست، تقویت سیگنال معنایی است – نه برداشتن دروازه‌ها.
- «خانه» داخل «کارخانه» تطبیق می‌خورد. بهای واقعی n-gram کاراکتری، که همان چیزی است که صرف فارسی را بدون ریشه‌یاب مدیریت می‌کند. عمداً اصلاح موردی نشده.
- بازیابی حداکثر ۴۰۰ کاندید پاس می‌دهد. برای پرسش‌های متنی و معنایی، `total` شمارش «آنچه قابل دسترسی است» است، نه هر آگهی پیکره که ممکن است تطبیق بخورد.
- اطمینان کالبره نشده. اطمینان ۰/۶ به این معنا نیست که ۶۰٪ برآوردها در بازه می‌افتند؛ یک عدد ترتیبی از عمق شواهد است. کالبراسیون واقعی نیازمند نمودار قابلیت اطمینان روی داده کنارگذاشته است.
- تشخیص تکراری فقط کارکرد و قیمت را می‌بیند. دو آگهی از یک خودرو با قیمت‌های به‌روز شده متفاوت (بیش از ۳٪) به هم پیوند نمی‌خورند. عمداً محافظه‌کارانه.

محدودیت‌های محصول

- خودروی صفرکیلومتر خارج از دامنه است – تصمیم آگاهانه به دلیل ساختار دوقیمتی بازار نو، با پرچم `--include-new` قابل بازگشت.
- بدون حساب کاربری، بدون ذخیره جست‌وجو، بدون هشدار قیمت. URL نقش نشانک را بازی می‌کند.
- SQLite برای یک نمونه است. برای چند نمونه با نوشتن همزمان، مهاجرت به PostgreSQL لازم است – اما این تصمیمی است که فقط زمانی معنا پیدا می‌کند که مسئله واقعی شود.

۱

کاتالوگ مشخصات فنی

بزرگ‌ترین برد قابل دسترس برای دقت قیمت: پرکردن حجم موتور، قدرت و مصرف برای هر برند+مدل+سال، مستقل از اینکه کدام سایت آگهی را منتشر کرده.

۲

کراول زمان‌بندی‌شده

تازه نگه‌داشتن پیکره و — مهم‌تر — ساختن سری زمانی که اجازه می‌دهد روند قیمت و سوگیری «توافقی» واقعاً اندازه‌گیری شود.

۳

اجرای لایه LLM

یک پاس دسته‌ای با همزمانی، که پس از آن کش رایگانش می‌کند و ظرفیت متن آگهی وارد امتیاز سلامت می‌شود.

۴

کالیبراسیون اطمینان

تبدیل عدد ترتیبی فعلی به احتمال واقعی پوشش، با نمودار قابلیت اطمینان روی داده کنارگذاشته.

۵

تقویت سیگنال معنایی

بازگرداندن دقت پرسش‌های مبهم به سطح خط‌مبنا، بدون از دست دادن دقت ۹۳٪ روی پرسش‌های عینی.

۶

منابع پنجم به بعد

قرارداد **Source** این را ارزان کرده: یک فایل جدید، دو متد، و هیچ تغییری در منطق محصول.

۲۲ پیوست: جدول کامل آمار پیکره

همه اعداد این جدول با پرس‌وجوی مستقیم از `backend/data/cars.db` در تاریخ گزارش استخراج شده‌اند.

حجم و ترکیب

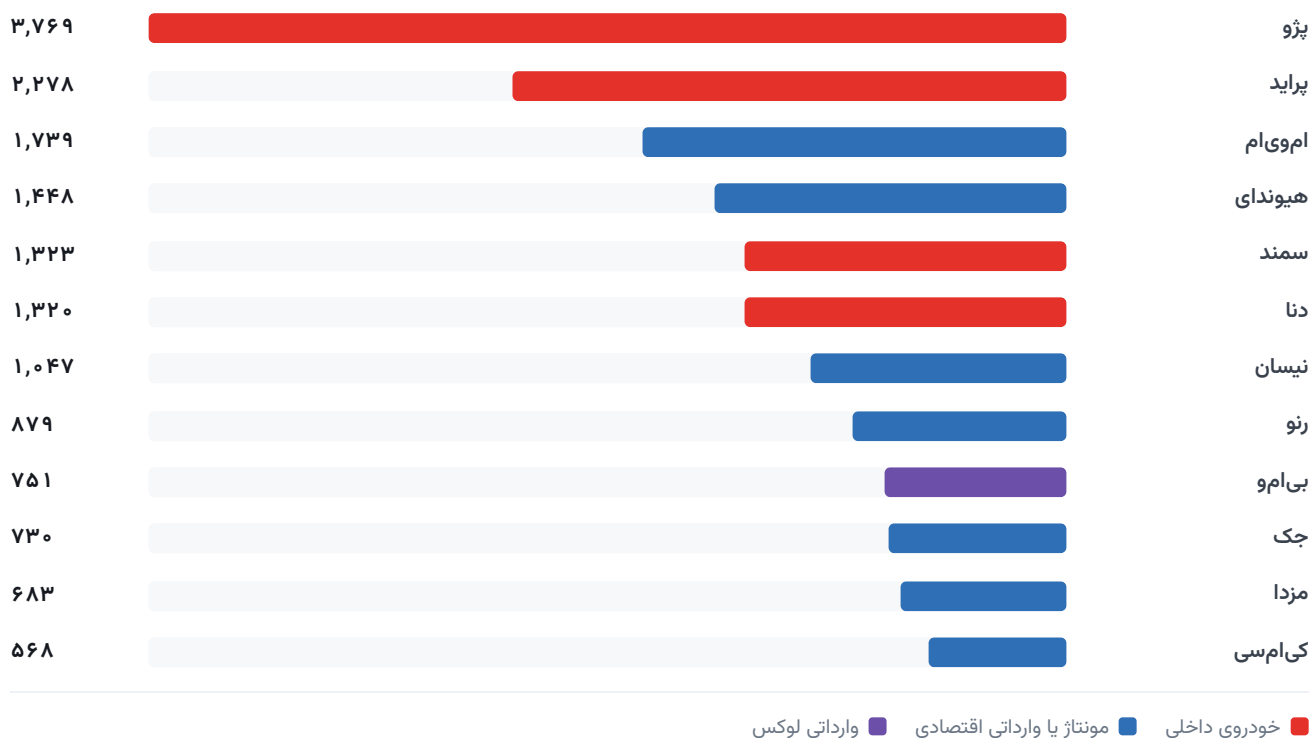
۲۲,۸۲۰	کل آگهی خودروی دست‌دوم
۴۷,۵٪ · ۱۰,۸۳۹	— شیپور
۴۲,۹٪ · ۹,۷۹۸	— باما
۸,۱٪ · ۱,۸۵۷	— دیوار
۱,۴٪ · ۳۲۶	— کارنامه
۸۸,۵٪ · ۲۰,۱۹۹	آگهی دارای قیمت منتشرشده
۱۱,۵٪ · ۲,۶۲۱	آگهی «توافقی» (بدون قیمت)
۹۹,۶٪ · ۲۲,۷۳۲	آگهی دارای برآورد قیمت
۲,۵۹۸	آگهی بدون قیمت که برآورد گرفت
۸۲۴	آگهی علامت‌خورده به‌عنوان ودیعه / حواله
۱,۰۵۸	آگهی شناسایی‌شده روی بیش از یک سایت
۲۱,۶۳۷	آگهی دارای ردیف غنی‌سازی
۱,۸۸۰	آگهی با دست‌کم یک پرچم ریسک

تنوع

۹۶	برند متمایز
۴۷۷	جفت برند + مدل متمایز
۵۶۳	شهر متمایز
۷۰	رنگ بدنه متمایز
۱۹۳	نام برند در واژگان جست‌وجو
۷۵۱	نام مدل در واژگان جست‌وجو
۱,۰۶۶	هم‌گروه سطح تیپ با ۵ مقایسه‌ای قیمت‌دار یا بیشتر

دوازده برند پرتکرار پیکره

اندازه‌گیری از ستون brand پس از حل هویت بین‌منبعی



ترکیب نوع فروشنده

تفکیک شخصی از نمایشگاه در امتیاز سلامت اثر مستقیم دارد



مدل قیمت – کارت خلاصه

HistGradientBoostingRegressor	الگوریتم
log(price_toman)	هدف
absolute_error	زبان
max_iter=400 · lr=0.06 · depth=7 · leaf=8 · l2=0.5	ابزارها
۸ عددی + ۸ دسته‌ای	ویژگی
۱۹,۲۸۱	سطر آموزش‌پذیر
۸۰ / ۲۰ با بذر ۴۲ – سپس برآزش دوباره روی کل داده	تقسیم
۱۴,۴%	MAPE کنارگذاشته
۷,۶%	خطای میانه کنارگذاشته
۰,۹۲۹	R ² روی لگاریتم
۰,۴۷۸	میانگین اطمینان پیکره

پشته فناوری

بک‌اند

Python ۳,۱۲ · FastAPI ۰,۱۱۵ · uvicorn ۰,۳۴ · scikit-learn ۱,۶ ·
pandas ۲,۲ · numpy ۲,۲ · httpx ۰,۲۸ · pydantic ۲,۱۰ · joblib ۱,۴ ·
openai ۱,۵۹ · SQLite · pytest ۹,۱

فرانت‌اند

· React ۱۹,۲ · TypeScript ۶,۰ · Vite ۸,۲ · Tailwind CSS ۳,۴ · oxlint
بدون کتابخانه UI، بدون کتابخانه نمودار، بدون مدیریت وضعیت خارجی

داده و انتساب

آگهی‌ها از نقطه‌های پایانی JSON عمومی و بدون احراز هویت Karnameh و Bama.ir، Divar، Sheypoor متعارف کراول شده‌اند، با تأخیر بین درخواست‌ها و کراولری که وقتی سایت می‌گوید آهسته‌تر، خودش را کند می‌کند. این یک دمو غیرتجاری است؛ تمام داده آگهی‌ها و تصاویر متعلق به همان سایت‌ها و فروشندگان اصلی باقی می‌ماند.

کاپوت · زیر کاپوتِ هر آگهی را ببین

پایان گزارش – ۲۲ بخش · تمام اعداد از کد و پیکره اجراشده استخراج شده‌اند